Data Pipeline Reference

Malay rojak lemmatiser — the **lemmatizer_lora_yt** adapter. The data behind it, how it was filtered and split, how the fine-tune works, and how it is evaluated.

```
cd unsloth
python chat.py --adapter lemmatizer_lora_yt
```

What this document is

Every number here was measured by running the code in the repository — re-running the corpus filter, replaying the inference call sequence with file access instrumented, and scoring the model against the gold set. None of it is quoted from documentation, because in several places the documentation and the code disagreed.

The repository has been reduced to this pipeline alone: the other two adapters and every dataset belonging to them have been removed (section 10). Section 3 is the one for a data collection chapter; section 8 covers evaluation; section 9 lists what belongs in limitations.

Contents

	Section	What it covers
1	**The pipeline at a glance**	four phases, and where weights change
2	**Terms that are easy to confuse**	labelling vs. learning; filtering vs. sampling
3	**Data collection**	source, method, yield by channel, and the write-up
4	**The funnel**	21,291 → 10,099 → 8,000 → 7,200
5	**The three-way split**	train 7,200 / dev 800 / test 150
6	**What data trained the model**	one file, and its composition
7	**The system: preprocessing and the model**	cleaning stages, base model, LoRA, hyperparameters, pseudocode
8	**Evaluation**	the metrics, the baselines, and the current results
9	**Things easy to miss**	for the limitations section
10	**What was removed**	and what each remaining file is for
11	**Report mapping and citations**	which table goes where
12	**Results**	tables, three-way comparison, system selection, algorithm analysis, coding

1 The pipeline at a glance

Four phases. Only the third changes any number inside the model, and the gold test set is read only in the fourth.

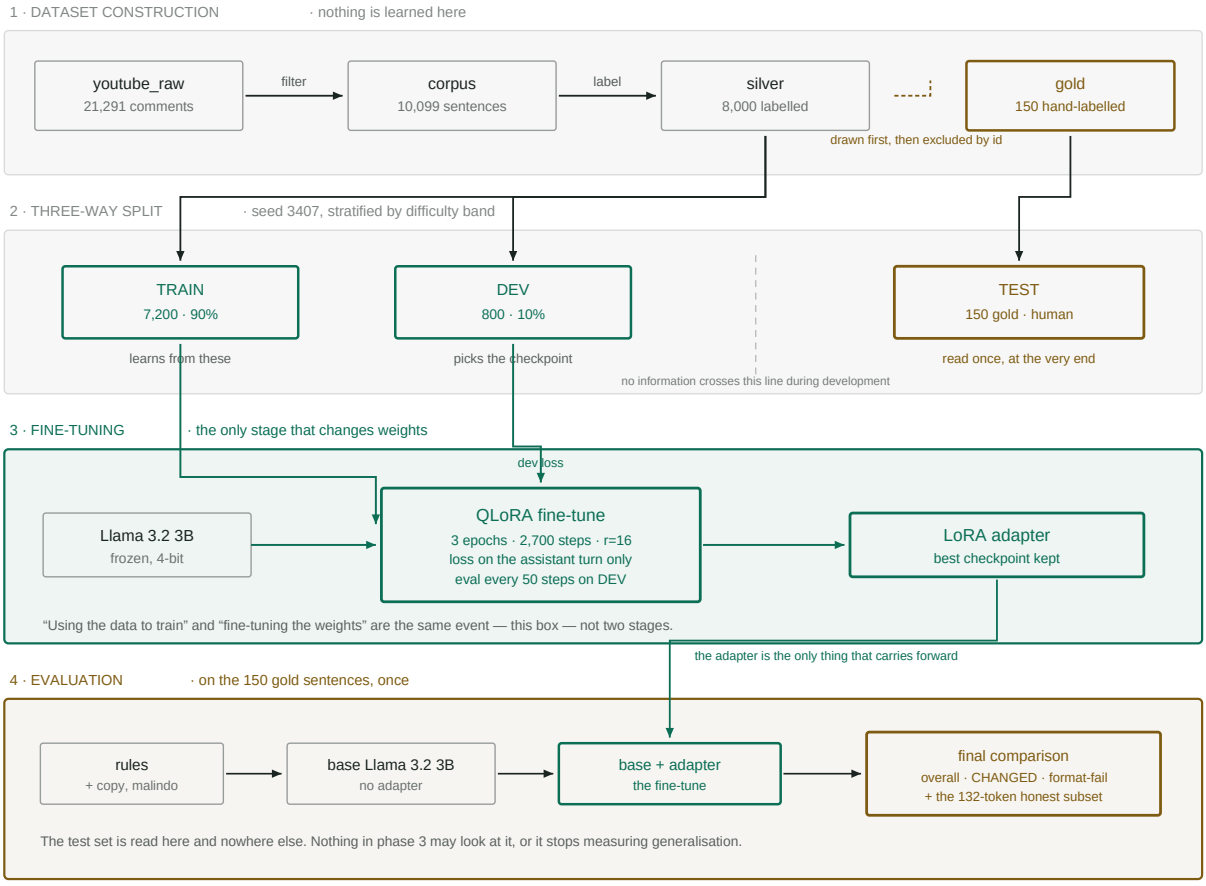


Figure 1. The 150 gold sentences are drawn before the silver data is sampled and excluded from it by id, so they can serve as a test set that nothing in training has seen.

2 Terms that are easy to confuse

Labelling is not learning

Two different things in this project lemmatise Malay, and only one of them learns. The **rule-based lemmatisers** — MALINDO, the SLANG and PARTICLES lexicons, orthographic repair — are lookup tables and string rules. They ran over the data in phase 1 to write down what the answer should be. Nothing about them improves; run them a hundred times and you get the same output. The **fine-tune** is a single QLoRA step that reads those answers and adjusts 97.3 MB of LoRA parameters. It is the only place gradient descent happens.

So "using the data to train" and "fine-tuning the weights" are the same event, not two stages.

Filtering is not sampling

Of the reductions in the funnel, only the first discards data for being unsuitable. The second is a sampling decision — a number that was chosen — and the third is a train/dev split in which nothing is discarded at all. Calling all three "filtering" would suggest roughly 18,000 sentences were unusable, when in fact about 7,000 perfectly good ones were simply never drawn.

3 Data collection

This section covers how the raw text was **obtained** — nothing else. Filtering, labelling, splitting and training each belong to their own chapter, and putting them here is the commonest way this section loses marks.

3.1 Collection parameters

Parameter	Value
Source	YouTube Data API v3, `commentThreads` endpoint
Channels sampled	9 Malaysian channels (podcast, lifestyle, news)
Videos attempted	180
Videos yielding comments	158
Cap per video	3 pages, approximately 300 comments
Comments collected	21,291 — 15,936 top-level, 5,355 replies
Comment date range	5 September 2025 – 18 August 2026
Fields retained	comment id, video id, text, likes, published_at, is_reply
Fields discarded	author display name, channel handle, profile URL
API cost	approximately 400 of the 10,000 free daily quota units

3.2 Yield by channel

Channel	Content type	Comments	Share
@safwannazripod	Podcast	8,396	39.4%
@ichangrm	Lifestyle	3,384	15.9%
@legasipodcast	Podcast	3,235	15.2%
@PaduBeb	Podcast	1,783	8.4%
@KumaarFamily	Lifestyle	1,744	8.2%
@thechiccodepodcast	Podcast	1,533	7.2%
@kitakitaajepodcast	Podcast	1,017	4.8%
@DESTINASITV	Travel	186	0.9%
@astroawani	News	13	0.1%
Total		**21,291**	**100%**

Keep the two smallest rows in the report. @astroawani returning 13 comments against a podcast's 8,396 is evidence for the sampling decision, not an embarrassment: news uploads frequently have comments disabled, and that is precisely why the corpus leans on podcasts.

3.3 A draft of the section

```
The corpus was collected from public YouTube comment sections using the
YouTube Data API v3. Nine Malaysian channels were sampled, weighted
towards podcast and lifestyle content: informal register, code-switching
and non-standard spelling occur far more densely there than in news
commentary, and a preliminary probe confirmed this before the full crawl.
Collection was capped at three pages per video (about 300 comments) to
avoid over-representing any single upload.
```

> Of 180 videos processed, 158 returned comments; the remainder had comments disabled. This yielded 21,291 comments (15,936 top-level and 5,355 replies) posted between September 2025 and August 2026, at a cost of roughly 400 of the 10,000 free daily API quota units.
>
> Only the comment text, its video identifier, like count, timestamp and reply flag were stored. Author display names, channel handles and profile URLs were discarded at collection time rather than removed later, so no identifying information about individual commenters is held in the dataset at any stage.

The final paragraph is worth keeping close to verbatim. Discarding identifiers *at collection* rather than anonymising afterwards is a stronger claim, and it is what the crawler actually does.

4 The funnel

The count goes **up** before it goes down. Each comment is first split into sentences, turning 21,291 comments into **38,821** candidate sentences. Five layers then run in order.

#	Layer	Test	Dropped	Why it is dropped
1	length	3 ≤ words ≤ 30	7,328	A one-word “betul” has no structure to lemmatise; a 60-word rant blows the sequence budget.
2	duplicate	normalised text seen before	2,389	Comment sections repeat — “first”, copied promotions, the same joke. Duplicates would be over-weighted.
3	script	≥ 80% Latin characters	18	Drops Chinese, Tamil and Arabic-script comments. Tiny, because the channels are Malay-language.
4	not Malay	≥ 2 Malay-evidence hits	11,868	**The biggest cut.** Pure English and emoji-only comments. Evidence counts function words *plus* slang, particles, x-negation and e-for-a spellings, because the most rojak sentences replace every standard function word.
5	not rojak	oov ≥ 2 or hits ≥ 4	6,926	Standard Malay, correctly spelled. Real text, but it teaches the model nothing: there is no rojak to resolve.
	Kept		**10,292**	Sentences dense enough in rojak to be worth labelling.

Layer 5's rule was tuned against the 150 hand-labelled sentences: `oov≥2 or hits≥4` scored 82% precision and 67% recall, against 70/78 for a flat `hits≥2`. Precision was bought deliberately — roughly 13,000 sentences were available and only 3,000 were needed, so discarding good rows is cheap while standard Malay leaking into the training set is not.

The file on disk says 10,099, not 10,292

Re-running the filter today, on the same 21,291 raw comments with the same code, keeps 10,292. The committed corpus.jsonl holds 10,099. The cause is the SLANG lexicon, which layer 4 counts as Malay evidence: it had 113 entries before the crawl commit and 136 after. Forcing it back to 113 yields 9,876; leaving it at 136 yields 10,292. The corpus was built at some point in between, while entries were still being added.

The exact figure is therefore not reproducible from any committed state. It is a small discrepancy, under 2%, but it is the honest kind of detail for a limitations section.

4.1 How the 3,000 are labelled

`make_goldsheet.propose()` assigns a lemma and a reason to every token, in a fixed order: numbers, then PARTICLES (31), then SLANG (136), then **BM_dict (590)**, then digit reduplication, then elongation, then MALINDO's root column, then its lemma column, and failing all of that the token is left alone and marked `oov`.

BM_dict was added to that chain deliberately. It holds 596 rojak → standard pairs of which most are absent from SLANG — `aq`→`aku`, `dlm`→`dalam`, `abeh`→`habis`. Four wrong entries were corrected in the process (`abg` mapped to *adik* rather than *abang*, and was teaching that to the model 41 times), and six vowel-dropped forms were added (`tdr`→`tidur`, `sngt`→`sangat`, `kne`→`kena`).

English detection was added last. The labeller previously had none, so every code-switched word fell through to `oov` — “nothing resolved it”. The most frequent unresolved tokens were `part` (264), `guest` (86), `please` (41). The label was already right — an English word in a Malay sentence is its own lemma — but the stated reason was wrong, which distorted every per-category table. A Hunspell `en_US` check now runs immediately before giving up, so those tokens are tagged `codeswitch`.

Label source	Tokens	Share
dict — MALINDO resolved it cleanly	24,174	25.0%
slang — SLANG or BM_dict expanded it	21,902	22.7%

Label source	Tokens	Share
ambiguous — several candidates, the first taken	12,466	12.9%
dict_check — MALINDO echoed an informal spelling back	9,682	10.0%
codeswitch — English, correctly left alone	8,041	8.3%
affix — a prefix or suffix was stripped	6,290	6.5%
oov — nothing resolved it	6,042	6.3%
particle, num, redup, elong	7,955	8.2%
unverified share	**34,480**	**35.7%**

The unverified share — oov, dict_check, ambiguous and affix, the four categories the labeller's own documentation marks “CHECK” — fell from **47.4% to 35.7%** across these changes. Most of that came from English detection: `oov` alone dropped from 14.6% to 6.3%.

SLANG is consulted first, so it keeps the 16 keys where the two lexicons disagree — `nak` stays *mahu* rather than BM_dict's *hendak*, `kat` stays *di* rather than *dekat*. That precedence matters: the 150 gold sentences were annotated against SLANG-derived pre-fills, so letting BM_dict override would put the training targets and the test labels out of step.

4.2 Sampling, not filtering: 10,099 → 3,000

This is `build_silver.py --n 8000`. Nothing is rejected for being unsuitable: **8,000 is a number that was chosen**, sampled stratified by difficulty band. The only exclusion is by identity — the 150 gold ids. The build refuses to run if it cannot find them to exclude, and asserts the exclusion afterwards. The remaining **1,949** sentences stay in corpus.jsonl, filtered and banded and unused.

The earlier version of this pipeline drew 3,000. It was raised to 8,000 for a specific reason: constructions the model was getting wrong were simply too rare at that size. The `menge-` prefix class, for instance, appeared 9 times in 3,000 sentences and 24 times in 8,000. Nine examples is not enough to learn a morphological rule from.

5 The three-way split

A fixed experimental design choice, not a tuned value. The purpose of the experiment is to determine whether LoRA fine-tuning improves Malay rojak lemmatisation over the unfine-tuned base model and the rule-based baselines — not to find an optimal ratio.

Split	Rows	Labels	What it is for	File
Train	7,200	silver (rule-generated)	The only data the optimiser sees.	unsloth/data/training_split.jsonl
Dev	800	silver (rule-generated)	Validation loss and checkpoint selection during training. Never used for the reported comparison.	unsloth/data/dev_split.jsonl
Test	150	human-verified	Read once, at the end, for the final comparison.	data/gold.jsonl

5.1 Stratification

Band	Train	Train %	Dev	Dev %
low	2,004	27.8%	223	27.9%
mid	2,852	39.6%	317	39.6%
high	2,344	32.6%	260	32.5%

Held out by band rather than by position, so the dev set carries the same difficulty mix as the training set and a dev loss is comparable across runs. Seed 3407, recorded in data/split_manifest.json along with every id.

5.2 What the design guarantees

```
python crawl/verify_splits.py

  train = 7200                                  PASS
  validation = 800                              PASS
  gold test = 150                               PASS
  train n validation = 0                        PASS
  train n gold = 0                              PASS
  validation n gold = 0                         PASS
  train u validation = silver ids               PASS
  train and dev share one system prompt         PASS
  band gaps < 3pp                               PASS
```

The dev set cannot compare rules against the fine-tune

Dev is silver-labelled, and the silver labels *are* the rule system's own output — so `rules` scores 100.0% on dev by construction. The only valid use of this split is monitoring the fine-tune during training. Every reported comparison belongs on the gold test set.

6 What data trained the model

`llm.py --train` opens exactly these. This is the complete list.

Resource	Type	Volume	How it contributes
unsloth/data/training_split.jsonl	ChatML dataset	7,200 rows	**The training data.** `train_on_responses_only()` masks the system and user turns, so loss is computed on the assistant turn alone.
unsloth/data/dev_split.jsonl	ChatML dataset	800 rows	Evaluated every 50 steps. Selects the checkpoint that is kept.
Llama-3.2-3B-Instruct-bnb-4bit	Pretrained weights	3B params	The frozen base, 4-bit. LoRA rank 16 adapts seven projections.
llama-3.2 chat template	Formatting spec	1	Renders each messages list into the token string, and defines the assistant header the loss mask keys off.

6.1 Anatomy of one row

```
{"messages": [
  {"role": "system",    ...}   masked - identical on all 7,200 rows
  {"role": "user",      ...}   masked - the rojak sentence, uncleaned
  {"role": "assistant", ...}   LOSS COMPUTED HERE - the lemma sentence
]}
```

6.2 Composition of the 2,700 rows

Property	Value	Note
Rows	7,200	90% of the 8,000 silver sentences
Distinct system turns	1	byte-identical to what chat.py sends at inference
Input words	86,651	mean 12.0, range 3–30
Target words	87,356	mean 12.1, range 3–37
Input characters	474,318	longest single input 241 characters
Target characters	483,680	longest single target 208 characters
Band: low / mid / high	2,004 / 2,852 / 2,344	27.8% / 39.6% / 32.6%
Source videos	130	comments dated September 2025 – August 2026
Mean difficulty score	8.70	range 3–42
Out-of-vocabulary tokens	17,214	mean 2.4 per sentence
Optimiser steps	2,700	7,200 rows × 3 epochs ÷ batch 8

Nothing is truncated: the longest row carries 208 characters of target against a 512-*token* window.

6.3 How much change each row asks for

Words changed between input and target	Rows	Share
None — target identical to input	379	5.3%
1–2 words	2,022	28.1%

Words changed between input and target	Rows	Share
3–5 words	2,225	30.9%
6–10 words	1,146	15.9%
11 or more	168	2.3%
Word count itself changed	1,260	17.5%

The last row is normalisation splitting or joining tokens — `xleh` → `tidak boleh`. It matters because a length change is exactly what makes token-level alignment fail at evaluation.

6.4 Rojak phenomena in the training text

Feature	Hits	Examples from the corpus
slang / abbreviation	15,484	mcm, yg, nk, sbb
vowel-dropping	4,712	tgk, smpii, dgn
discourse particle	3,370	la, kot, weh, doh
code-switching	1,599	renovate, guest, story
reduplication	1,330	budak2, cantik cantik
elongation	1,006	gilaaa, okaayyyy
e-for-a spelling	372	ape, kenape, die
x-negation	305	xleh, xde, xjadi

7 The system: preprocessing and the model

7.1 The preprocessing stage

Rojak comment text is not clean input. A single sentence can carry emoji, a URL, an @handle, digits glued onto words (`8pagi`), letters repeated for emphasis (`gilaaa`) and the digit “2” used as a reduplication marker (`budak2`). `unsloth/preprocess.py` normalises all of that in four ordered stages before the sentence reaches the model. It is stdlib-only and GPU-free, so it can be tested on its own with `python preprocess.py --demo`.

Stage	What it does	Worked example
input	—	Budak2 tu gilaaa!! 3 org je *[2 emoji]*
1 — characters	Lowercase, then collapse a letter repeated three or more times, per run of letters rather than per sentence	budak2 tu gila!! 3 org je *[2 emoji]*
2 — reduplication	**Preserves the digit-2 marker.** Expansion is available but off by default — see the note below	budak2 tu gila!! 3 org je *[2 emoji]*
3 — elimination	NFKC normalise; drop everything at codepoint 0x2E80 and above (emoji, CJK); remove URLs and @handles; apply the number policy; keep only a–z, space, apostrophe and hyphen	budak2 tu gila org je
4 — tokenisation	Split with the same pattern that tokenised the corpus, so a token means here what it meant at annotation time	['budak2', 'tu', 'gila', 'org', 'je']

Stage 3 must not touch the marker either. Whatever the digit policy does to real numbers, the “2” in `budak2` has to survive it — strip it and the word silently becomes a different one, with no trace that anything was lost. It is handled per token, so the policy still applies normally everywhere else.

Digits are the awkward case, so the policy is a flag:

Policy	Rule	"bayar rm3.50 je pukul 8pagi 2019" becomes
strip *(default)*	remove digit characters, keep the letters around them	bayar rm je pukul pagi
drop	remove any whitespace token containing a digit	bayar je pukul
keep	leave digits alone	bayar rm3 50 je pukul 8pagi 2019

strip is the default because rojak glues counts onto real words — `3hari`, `8pagi` — and dropping the whole token would throw away the word the lemmatiser actually needs. The cost is visible above: a token that is mostly digits leaves a stub, so `rm3.50` becomes `rm`. Use **drop** if that matters more than keeping *pagi*.

What preprocessing deliberately does not do

No slang expansion, no affix stripping, no dictionary lookup. Those are the model’s job — they are what the fine-tune exists to perform. Doing them here would double-handle exactly the tokens the adapter was trained on, and would make it impossible to say which component produced a given lemma. The module normalises *orthography*; the model does *lemmatisation*.

Why stage 2 was turned off

This module used to **expand** reduplication (`budak2` → `budak-budak`) while the labeller **resolves** it to the base. The training data settles which is right: **419 of the training inputs carry the digit form and only 22 carry the hyphenated one**, so expanding handed the model the rare spelling of a construction it knows well in the common one.

It also broke the output. `budak-budak` is a single token to the tokeniser, the model answers with two words, and one token against two words failed alignment outright — the user saw *"1 tokens in, 2 words out, cannot be aligned"* and no rule table at all. The marker is now preserved end to end, and `align_lemmas()` additionally tolerates the hyphenated form mapping to two words, so a directly typed `rama-rama` still produces a table.

Preprocessing shifts the input away from training

The adapter was fine-tuned on **uncleaned** text: the user turns in the training file still carry capitals, commas and digits. Cleaning the input at inference therefore moves it away from what the model saw during training. It is still usually the right call for messy input, but the honest fix is to run the same preprocessing over the training data and retrain, so the two sides match.

`chat.py --raw` skips preprocessing entirely, and is the only way to compare the two fairly.

7.2 The base model

Property	Value	Why it was chosen
Model	Llama 3.2 3B Instruct	Decoder-only transformer, already instruction-tuned, so it follows a system prompt without further work.
Distribution	unsloth/Llama-3.2-3B-Instruct-bnb-4bit	Pre-quantised, so no conversion step and no full-precision download.
Parameters	approximately 3.2 billion	The largest size that trains in 8 GB of VRAM once quantised.
Precision	4-bit (bitsandbytes NF4)	Weights are stored in 4 bits and dequantised on the fly during the forward pass; this is the QLoRA idea.
Context used	512 tokens	Rojak comment sentences are short — the longest training row is 190 characters of target.
Chat template	llama-3.2	Defines the role headers, including the assistant header the loss mask keys off.

7.3 What LoRA actually changes

Full fine-tuning would update every one of the 3.2 billion weights, which neither fits in memory nor is necessary. LoRA leaves the original weight matrix W frozen and learns a low-rank correction beside it. For a layer whose weights are $d \times k$, it trains two much smaller matrices — B of size $d \times r$ and A of size $r \times k$ — and uses $W + BA$ at inference. With $r = 16$ against a hidden size of 3,072, those two matrices are a small fraction of the original.

Reading the shapes straight out of the saved adapter gives the exact cost:

Quantity	Value
LoRA tensors saved	392 (an A and a B per adapted layer)
Trainable parameters	**24,313,856**
As a share of the base model	**0.76%**
Stored size	97.3 MB in float32

Quantity	Value
Rank *r* / scaling *alpha*	16 / 16, so the update is applied at scale alpha÷r = 1
Dropout	0

Spread across the seven adapted projections:

Projection	Role in the block	LoRA parameters
gate, up, down	the MLP / feed-forward path	5,046,272 each
q, o	attention query and output	2,752,512 each
k, v	attention key and value	1,835,008 each

Two consequences worth stating in a report. First, the artefact this project produces is **not a model** — it is a 97 MB patch, meaningless without the base weights, which is why `chat.py` loads both. Second, because the base is frozen, whatever Malay the model already knew is preserved rather than overwritten; the adapter adds a task, it does not replace the language.

7.4 Loss masking

`train_on_responses_only()` masks everything up to the assistant header. The model receives the full conversation but is scored only on producing the lemma sentence, never on reproducing the system prompt or the user's sentence. Without this, most of the gradient would be spent teaching it to echo the input.

7.5 Hyperparameters

Setting	Value	Note
Base model	unsloth/Llama-3.2-3B-Instruct-bnb-4bit	4-bit quantised
LoRA rank (r) / alpha / dropout	16 / 16 / 0	—
Target modules	q, k, v, o, gate, up, down	all attention + MLP projections
max_seq_length	512	longest row is well inside this
Batch size × grad accumulation	2 × 4	effective batch 8
Epochs	3	—
Optimiser steps	2,700	7,200 × 3 ÷ 8
Learning rate	2e-4	linear schedule, 5 warmup steps
Optimiser	adamw_8bit	weight decay 0.01
Seed	3407	—
Evaluation	every 50 steps on the dev split	54 evaluations across the run
Checkpointing	every 50 steps, keep 2	load_best_model_at_end on eval_loss

The evaluation and checkpoint cadences must match for `load_best_model_at_end` to work. llm.py builds its config through a helper that asks the installed TRL and transformers what arguments they accept and **prints anything it drops** — a silently ignored eval_strategy is how you end up believing you have validation when you do not.

7.6 The prompt contract

The system turn is identical on all 7,200 rows and must stay byte-identical between training and inference. It is not retyped anywhere: `skill.prompt_for()` asks the adapter which prompt it needs.

```
Anda adalah pelemma bahasa Melayu. Tukarkan ayat rojak kepada
ayat berbentuk lema. Kembalikan ayat sahaja.
```

7.8 System pseudocode

Four algorithms describe the system end to end: how an input sentence is cleaned, how the model turns it into a lemma sentence, how each token is then explained, and how the adapter was produced in the first place. Together they are what `chat.py --adapter lemmatizer_lora_yt` and `llm.py --train` actually do.

```
ALGORITHM 1   Preprocess(text, policy)      clean a rojak sentence
-----------------------------------------------------------------
Input   text     raw user sentence
        policy   strip | drop | keep        what to do with digits
Output  s        cleaned sentence, tokens joined by single spaces

 1  t <- lowercase(text)                        STAGE 1  characters
 2  for each maximal run of letters r in t do
 3      t <- t with r replaced by CollapseRepeat(r)   gilaaa -> gila

 4  for each whitespace-separated word w in t do  STAGE 2  reduplication
 5      if w matches ^(base)2(tail)$ then
 6          w <- base + '-' + base + tail          budak2 -> budak-budak

 7  t <- NFKC(t)                                  STAGE 3  elimination
 8  t <- drop characters with codepoint >= 0x2E80    emoji, CJK
 9  t <- remove URLs and @handles
10  if policy = drop  then remove any word containing a digit
11  if policy = strip then remove digit characters only  8pagi -> pagi
12  t <- remove all but  a-z, space, apostrophe, hyphen
13  t <- collapse runs of whitespace

14  T <- TokenPattern.findall(t)                 STAGE 4  tokenisation
15  return join(T, ' ')
```

The stage order is load-bearing and not arbitrary. Stage 2 must precede stage 3, because the “2” in `budak2` is a reduplication marker, not a number — eliminate digits first and it silently becomes *budak*, a different word with no trace that anything was lost. Stage 1 must precede stage 2 for the same class of reason: `budakk2` is an elongated reduplication, and only collapsing the doubled *k* first lets stage 2 recognise it.

```
ALGORITHM 2   Lemmatise(text)              the inference path
-------------------------------------------------------------------
Input   text     raw sentence typed by the user
Output  answer   the lemma sentence
        R        one (token, lemma, rule, confidence) row per token

 1  prompt, version <- PromptFor(adapter)
 2      read adapter/skill.json if present, else skill.ADAPTERS by
 3      folder name.  NEVER retype the prompt: it must be byte-
 4      identical to the system turn the adapter was trained on.

 5  s <- Preprocess(text)                    skipped when --raw
 6  if s is empty then
 7      report 'nothing left after preprocessing' and stop
 8      an empty user turn does not give an empty answer -- it makes
 9      the model invent a sentence to lemmatise

10  msgs <- [ (system, prompt), (user, s) ]
11  ids  <- ChatTemplate(msgs, add_generation_prompt = true)
12  out  <- Generate(model, ids, max_new_tokens = 256,
13                   do_sample = false)          greedy, deterministic
14  answer <- Decode( out[ |ids| .. ] )       drop the echoed prompt
```

```
15  T <- split(s)                            tokens sent to the model
16  A <- AlignLemmas(T, split(answer))
17      widths from SLANG: a token whose expansion is k words claims
18      k output words, so 'xleh' -> 'tidak boleh' does not shift the
19      rest of the row.  Falls back to 1:1 when the totals allow it.

20  if A = null then
21      print answer and the two token counts, and stop
22      an unreadable answer is a result; hiding it costs the one
23      clue that the adapter and the prompt disagree

24  for i <- 1 to |T| do
25      rule_i, uncertain_i <- DeriveRule(T_i, A_i)
26  return answer, [ (T_i, A_i, rule_i, uncertain_i) ]
```

Line 25 is what makes the printed rule column honest. A v1 adapter answers with a bare lemma sentence and states no reasoning, so the rules are reconstructed here, after the fact, and marked `[nota]` in the output so they are never mistaken for the model's own explanation.

```
ALGORITHM 3   DeriveRule(t, l)          why did token t become l ?
-----------------------------------------------------------------------
Input   t   the input token,   l   the lemma the model returned
Output  one of nine tags, and whether it is a demonstrated account
        or merely the best available one

  ordered strongest evidence first; FIRST MATCH WINS, so this order
  IS the precedence

 1  if t matches money | time | date | percent | number
 2                                    return NOMBOR,   certain
 3  if t = 'x-x' for some x, or t matches base+'2'
 4                                    return GANDA,    certain
 5  if CollapseRepeat(t) = l          return PANJANG,  certain
 6  if t in PARTICLES and t = l       return PARTIKEL, certain
 7  if t in SLANG                     return RINGKAS,
 8                                       uncertain if SLANG[t] != l
 9  if MALINDO lists l as a root of surface t, and t != l
10                                    return IMBUHAN,  certain
11  if t != l and ( StripAffix(t) = l or LooseStrip(t) = l )
12                                    return IMBUHAN,  certain

13  if t = l then                           nothing changed
14      if t in EN_HINT or t in EN_VOCAB
15                                    return INGGERIS, certain
16      if t in MALINDO Kamus roots or t in ROOTS
17                                    return KAMUS,    certain
18      return KEKAL, UNCERTAIN
19          an English word the hint list lacks is indistinguishable
20          from a Malay word with no known root: both were kept

21  if t contains an elongated run    return PANJANG,  uncertain
22  return RINGKAS, uncertain               changed, unexplained
```

Line 14 tests English *before* roothood because the two overlap asymmetrically: plenty of English words appear somewhere in a Malay corpus, whereas EN_VOCAB holds only tokens the sentiment corpus saw overwhelmingly in English lines. When both fire, English is the better-evidenced of the two.

```
ALGORITHM 4   FineTune(D_train, D_dev)      how the adapter is made
-----------------------------------------------------------------------
```

```
Input   D_train  2,700 ChatML rows,   D_dev  300 ChatML rows
Output  a LoRA adapter, stamped with its skill version

 1  M, tok <- LoadBase(Llama-3.2-3B-Instruct, 4-bit,
 2                     max_seq_length = 512)
 3  M      <- AttachLoRA(M, r = 16, alpha = 16, dropout = 0,
 4                       targets = {q,k,v,o,gate,up,down})
 5      the 3B base weights are FROZEN; only the new low-rank
 6      matrices receive gradient
 7  tok    <- ApplyChatTemplate(tok, 'llama-3.2')

 8  for each split S in {D_train, D_dev} do
 9      S.text <- ChatTemplate(S.messages)     one string per row
10  mask every token up to the assistant header
11      loss is computed on the lemma sentence alone, never on the
12      prompt or the user's sentence

13  best <- +infinity ;  best_step <- 0
14  for step <- 1 to 1012 do          3 epochs at effective batch 8
15      B <- next batch from D_train
16      update the LoRA weights by AdamW-8bit on loss(M, B)
17      if step mod 50 = 0 then
18          L <- MeanLoss(M, D_dev)            the dev split earns
19          save checkpoint at step            its place here
20          if L < best then best <- L ; best_step <- step

21  M <- checkpoint at best_step               load_best_model_at_end
22  save M and its tokenizer, and stamp skill version 'v1'
23  D_gold is never read anywhere in this procedure
```

Line 23 is the property the whole three-way design exists to guarantee, and `crawl/verify_splits.py` asserts it against the files on disk.

7.7 Constants that look authoritative but do not apply

Several constants are defined more than once, with different values, because the repository once hosted three pipelines. Reading the wrong one is the easiest way to put a false number in a report.

Constant	Value for this pipeline	Other definitions still in the repository
VAL_RATIO / DEV_RATIO	**0.10** — crawl/build_silver.py, which performs the 7,200 / 800 split	**0.15** in unsloth/llm.py and unsloth/sources.py. Neither applies: llm.py's is used only by `--prepare` on the missing seed500.jsonl, and sources.py belongs to the v2 pipeline whose data has been removed. The arithmetic settles it — 8,000 × 0.10 = 800, the size of dev_split.jsonl.
SEED_FILE	not used — build_silver.py supplies the data	`seed500.jsonl` in llm.py. That file is not on disk, so `llm.py --prepare` cannot run in its v1 branch.
MAX_SEQ_LEN	**512** — the v1 branch of llm.py and chat.py	**1024** for v2, whose reasoning targets run 150–300 tokens against v1's ~30.
SYSTEM_PROMPT	**skill.SYSTEM_PROMPT_V1** — the prompt in 7.6	`skill.SYSTEM_PROMPT` without the suffix is the *v2* reasoning prompt. llm.py chooses on the `USE_REASONING` environment variable, which describes a training run rather than the model being loaded. Always export `USE_REASONING=0`.
SLANG	**136 entries**, databuilder.SLANG	crawl/build_corpus.py carries its own inlined copy. Identical today, verified, but duplicated rather than shared, so they can drift.

A quick way to check any constant: ask which file *wrote the artefact you are describing*, and read the constant from that file.

8 Evaluation

Scored by `crawl/evaluate.py` on the 150 gold sentences: 1,786 tokens, of which 595 (33%) need changing.

```
python crawl/evaluate.py --model --base   # one GPU pass, fills every table
python crawl/evaluate.py --reuse --base   # re-score the cache, no GPU
python crawl/evaluate.py --dev --model    # development only
```

8.1 The metrics

Metric	What it measures	Why it is there
overall	token accuracy over all 1,786 gold tokens	Dominated by the 67% of tokens that are already their own lemma.
CHANGED	accuracy on the 595 tokens whose lemma differs from the surface form	The actual work. `copy` scores 0% here by definition.
unchanged	accuracy on the tokens that needed no change	Catches a model that over-corrects.
sent-exact	whole-sentence exact match	Strict, and sensitive to a single wrong token.
format-fail	share of sentences whose output length does not match the input	A model that will not hold the format. Scored by LCS alignment rather than zeroed, so the accuracy columns stay meaningful.
precision / recall / F1	the decision to change a token, as a confusion matrix	Recall is the CHANGED column renamed; precision is what accuracy hides — over-correction.
WER / CER	corpus-level word and character edit distance	Partial credit for near misses, and sensitive to insertions the alignment absorbed.
McNemar p	paired significance against the rules baseline	Turns a gap in the table into a claim that can be defended.
honest subset	accuracy on the 132 tokens a human overrode	**The one that matters.** The gold labels were pre-filled by the rule system, so on the other 1,654 tokens the annotator largely accepted rule output and `rules` scores itself correct. Only the overridden tokens are independent of it.

8.2 Current results on the gold test set

System	Overall	CHANGED	Unchanged	Sent-exact	Format-fail
copy	66.7%	0.0%	100.0%	4.0%	n/a
malindo	66.9%	0.8%	99.8%	4.7%	n/a
dict-root	74.2%	22.5%	100.0%	6.7%	n/a
rules	95.1%	88.4%	98.5%	64.7%	n/a
base Llama 3.2 3B	—	—	—	—	—
fine-tuned	—	—	—	—	—

The two neural rows are empty because they need a GPU pass: `python crawl/evaluate.py --model --base` fills both. The rule baselines are already the post-merge figures. Note that `rules` gained from the BM_dict merge too — CHANGED rose from 83.0% to 88.2% — because the same lexicon strengthens the baseline and the training targets alike. That is the fair comparison.

8.3 The honest subset — 132 human-overridden tokens

System	Accuracy	Reading
copy	3.0%	the floor
malindo	1.5%	a plain dictionary cannot touch these
dict-root	3.0%	nor can the root column
rules	47.0%	the strongest non-neural system
fine-tuned	—	to be measured on the retrained adapter

This is the table that decides whether the project worked. On the headline “overall” column a fine-tune can lose to `rules` without that meaning anything, because gold was pre-filled by the rule system and it scores itself correct on the 1,654 tokens the annotator accepted. Only these 132 are independent of it.

The bar has moved, twice. Before BM_dict was added to the labeller the rule baseline scored 22.0% here and an earlier adapter scored 54.5% — a gap that flattered the model, because that adapter had been trained on BM_dict-flavoured targets while the baseline was denied the same lexicon. Giving both the same lexicon, and then correcting four wrong entries in it, brings the baseline to **47.0%**. That is the honest number to beat.

8.4 Classification and sequence metrics

Token accuracy answers “how often is the lemma right”. It does not answer “how often does the system change something it should have left alone”, and on this task that second question matters as much as the first. Framing lemmatisation as a decision to **change** a token gives a confusion matrix and the three standard measures.

Cell	Meaning
TP	the token needed changing, the system changed it, and got it right
FN	the token needed changing and the system left it — or changed it wrongly
FP	the system changed a token that needed no change (**over-correction**), or changed one wrongly
TN	the token needed no change and none was made

Recall is the CHANGED column under another name, so the two are never reported as if they were independent evidence. Precision is the genuinely new number, and F1 is the honest single figure.

Measured on the gold test set:

System	TP	FP	FN	TN	Precision	Recall	F1
copy	0	0	595	1,191	0.0%	0.0%	0.0%
malindo	5	23	590	1,189	17.9%	0.8%	1.6%
dict-root	134	3	461	1,191	97.8%	22.5%	36.6%
rules	526	22	69	1,173	96.0%	88.4%	92.0%
base Llama	—	—	—	—	—	—	—
fine-tuned	—	—	—	—	—	—	—

`dict-root` shows why precision earns its place: 97.8% precision against 22.5% recall means it is cautious and almost always right when it acts. Accuracy alone cannot express that.

Word and character error rate

Both are edit distance against the gold lemma sentence, accumulated over the corpus rather than averaged per sentence — otherwise a three-word sentence would weigh as much as a thirty-word one. For the neural systems the hypothesis is the **raw generation**, not the aligned tokens, so words the alignment absorbed still

count as insertions or deletions; otherwise WER would forgive precisely the format failures it exists to catch.

System	WER	CER
copy	0.3396	0.1699
malindo	0.3380	0.1750
dict-root	0.2653	0.1284
rules	0.0638	0.0244
base Llama	—	—
fine-tuned	—	—

CER gives partial credit where token accuracy scores a flat zero: `tengo` for `tengok` is one character wrong, not a whole token wrong. It is the more informative of the two here, because rojak lemmatisation is largely character-level surgery.

Statistical significance

Two systems are scored on the *same* tokens, so their outcomes are paired and only the disagreements carry information. **McNemar's exact test** is the appropriate one, and it is reported against the `rules` baseline for every other system, on both the full set and the 132-token honest subset.

Comparison	Subset	rules wins	p	Verdict
copy vs rules	1,786 tokens	526	7.4e-131	significant
malindo vs rules	1,786 tokens	521	8.4e-132	significant
dict-root vs rules	1,786 tokens	393	5.2e-92	significant
dict-root vs rules	132 honest	60	8.5e-16	significant

A small p says the gap is unlikely to be chance. It does not say the gap is large — that is what the confidence intervals below are for.

8.5 Formal definitions

Let the test set be G, the 150 gold sentences. A sentence s has tokens $t_1 \ldots t_n$ with human-verified lemmas $l_1 \ldots l_n$. A system produces one prediction p_i per token, which may be *null* when nothing could be aligned to that position.

Comparison is on a normalised form, $norm(w) = w$ lowercased with non-word characters removed, so that punctuation and casing are not counted as lemmatisation errors. Two indicators are defined per token:

```
correct(i) = 1  if p_i is not null and norm(p_i) = norm(l_i)
             0  otherwise

changed(i) = 1  if norm(t_i) != norm(l_i)      the token needs work
             0  otherwise                      already its own lemma
```

Metric	Definition	n
Overall accuracy	Σ correct(i) ÷ Σ 1, over every token in G	1,786
CHANGED accuracy	Σ correct(i)·changed(i) ÷ Σ changed(i)	595
Unchanged accuracy	Σ correct(i)·(1–changed(i)) ÷ Σ (1–changed(i))	1,191
Sentence exact match	fraction of sentences where correct(i) = 1 for every i	150
Format-failure rate	fraction of sentences where the generated word count differs from Σ $\lvert l_i \rvert$ (the expected count)	150

Metric	Definition	n
Honest-subset accuracy	overall accuracy restricted to tokens whose gold annotation carries `src = human`	132

The denominators differ, and that is the point. Overall accuracy is dominated by the 1,191 tokens that were already their own lemma, where returning the input unchanged scores 100%. CHANGED accuracy isolates the work. The honest subset isolates the tokens on which the rule system was demonstrably wrong.

8.6 Aligning a generated sentence to gold tokens

The rule baselines emit one prediction per token, so scoring them is direct. The neural systems emit a *sentence*, and it has to be matched back to the token positions before anything can be counted. Two complications make this non-trivial.

- **A lemma may be several words.** `xleh` becomes *tidak boleh*, so the expected word count is $\Sigma\,|l_i|$, not n.
- **A generative model may drop or insert a word anywhere.** Under purely positional matching a single spurious word shifts every subsequent token and the whole sentence scores zero — which measures formatting, not lemmatisation.

The evaluation therefore uses a two-stage alignment. Positional matching is attempted first and is exact when the counts agree; otherwise the sentence is recorded as a **format failure** and scored by longest common subsequence, so that each token is still judged on its own merits. Reporting the format-failure rate alongside the accuracies keeps the two failure modes separable: a model that lemmatises well but will not hold the output format is a different problem from one that holds the format and lemmatises badly.

8.7 Pseudocode

```
ALGORITHM 5   Align(P, L)   generated words -> gold token positions
-----------------------------------------------------------------------
Input   P = w_1..w_m        words of the generated sentence
        L = l_1..l_n        gold lemmas, each 1 or more words
Output  A = a_1..a_n        matched prediction per token, or null
        strict              true if positional alignment succeeded

 1  expected <- SUM over i of |l_i|           expected word count
 2  if m = expected then                      STRICT alignment
 3      j <- 1
 4      for i <- 1 to n do
 5          k    <- |l_i|
 6          a_i  <- join(w_j .. w_{j+k-1})
 7          j    <- j + k
 8      return A, strict = true

 9  else                                      LCS alignment
10      flatten L into gold words g_1..g_expected,
11         recording owner(q) = the token index each g_q came from
12      B <- MatchingBlocks( norm(g), norm(w) )   longest common
13      for each block (start_g, start_w, size) in B do
14          for d <- 0 to size-1 do
15              q <- start_g + d ;  r <- start_w + d
16              mark g_q matched
17              append w_r to a_{owner(q)}
18      for i <- 1 to n do                    all-or-nothing rule
19          if any word of l_i is unmatched then a_i <- null
20      return A, strict = false
```

Line 19 of Algorithm 5 is deliberate: a multi-word lemma counts only if every one of its words aligned, so a system that produces *tidak* where *tidak boleh* was required receives no credit.

```
ALGORITHM 6   Score(G, f)   evaluate one system on the gold set
-----------------------------------------------------------------------
Input   G      gold sentences with per-token annotation
        f      a system: rule baseline, or a generative model
Output  the metrics of section 8.5

 1  for each counter c in {all, changed, same, exact, human}
 2      hit[c] <- 0 ;  tot[c] <- 0
 3  fail <- 0

 4  for each sentence s in G do
 5      if f is a rule baseline then
 6          A <- f(s)                    one prediction per token
 7      else
 8          text     <- Generate(f, SystemPrompt, s.text)
 9          A, strict <- Align( split(text), s.lemmas )
10          if not strict then fail <- fail + 1

11      ok_sentence <- true
12      for i <- 1 to |s.lemmas| do
13          hit_i <- ( a_i != null and norm(a_i) = norm(l_i) )
14          bucket <- changed  if norm(t_i) != norm(l_i)
15                    same     otherwise
16          for c in {all, bucket} do
17              tot[c] <- tot[c] + 1 ;  hit[c] <- hit[c] + hit_i
18          if src(t_i) = human then
19              tot[human] <- tot[human] + 1
20              hit[human] <- hit[human] + hit_i
21          ok_sentence <- ok_sentence AND hit_i
22      tot[exact] <- tot[exact] + 1
23      hit[exact] <- hit[exact] + ok_sentence

24  return  hit[c] / tot[c]  for each c,  and  fail / |G|
```

Generation is greedy (`do_sample = False`) so the procedure is deterministic and a cached prediction file can be re-scored without re-running the model. The base model and the fine-tuned model receive the identical system prompt; only the weights differ.

8.8 The systems compared

System	Prediction rule for token t	Why it is included
copy	return *t* unchanged	The floor. 67% of gold tokens are already their own lemma, so any system below this is worse than doing nothing.
malindo	MALINDO lemma column; choose *t* if it is itself listed, else the shortest candidate	A plain dictionary lemmatiser — what a non-ML pipeline gives for free.
dict-root	MALINDO *root* column, falling back to the lemma column	Fairer to the dictionary: the root column strips affixes properly, where the lemma column often echoes the surface form.
rules	PARTICLES → SLANG → BM_dict → reduplication → elongation → MALINDO root → MALINDO lemma → unchanged	The strongest system buildable without machine learning, and the one the fine-tune must beat to justify itself.
base Llama 3.2 3B	generate, then align	Isolates what the prompt alone achieves, with no task-specific training.

System	Prediction rule for token t	Why it is included
fine-tuned	generate with the LoRA adapter applied, then align	The contribution under test.

The first four are deterministic functions of the token and the lexicons, so they need no GPU and are recomputed on every run. The last two are cached per system, under separate filenames, so that predictions from one model can never be scored as another's.

How much of a difference would be real

With n=132 the 95% confidence interval on the honest subset is ±8.5 percentage points, and the smallest gap between two runs that could be called real is about **12 points**. On the CHANGED subset (n=595) it is about 4 points. Differences smaller than that are noise, whatever the table says.

Two consequences. Run 2–3 seeds per configuration and report the spread, because seed variance may exceed the effect being measured. And if a final margin looks close, labelling another 150 gold sentences buys more than any amount of tuning.

9 Things easy to miss

Findings that are not visible from the filenames, each of which belongs somewhere in a methodology or limitations section.

9.1 No human checked any training label

The rule-based labeller flags **17,029 of 35,950 tokens (47.4%)** as unverified on its own admission — the categories oov, dict_check, ambiguous and affix, which its own documentation marks “CHECK”. That figure is a ceiling on what a model trained on these labels can be. Stating it is stronger than leaving a reader to find it.

9.2 The rules baseline is contaminated

Gold annotation fed back into the SLANG lexicon: it grew from 113 to 136 entries in the same commit that added gold.jsonl, and **14 of the 23 new entries are tokens a human corrected there**. The `rules` baseline is MALINDO + SLANG + orthographic repair, scored on those same 150 sentences. **43 of 1,786 gold tokens (2.4%), across 36 of the 150 sentences**, are now covered by entries derived from that annotation. It does not touch the fine-tune's own score, but it inflates the baseline the fine-tune is compared against.

9.3 The model was trained on uncleaned text

The user turns still carry capitals, commas and digits. chat.py cleans the input by default, which shifts it away from what the model saw. Compare with `--raw` before trusting either. The honest fix is to run the same preprocessing over the training targets and retrain, so the two sides match.

9.4 Only a quarter of the available data is used

10,099 sentences survived the filter; 3,000 were labelled; 2,700 train the model. The other 6,949 are filtered, banded and unused — the obvious lever for a larger run, and a better experiment than varying the split ratio.

9.5 The corpus is not reproducible from a committed state

Re-running the filter today keeps 10,292 sentences against the 10,099 on disk, because the SLANG lexicon was still being edited when the corpus was generated. Under 2%, but worth one sentence.

9.6 The v1 output format is unauditable on its own

A `v1` adapter returns a bare lemma sentence. When it returns `jalan` for `berjalan` you cannot tell whether it stripped a prefix, hit the lexicon, or guessed. The rule tags chat.py prints are re-derived afterwards and marked `[nota]` so they are never mistaken for the model's own reasoning.

9.7 Three epochs is slightly too many

The dev set earns its place immediately. Across 21 evaluations the validation loss fell from 0.2454 to **0.0637 at step 700**, then drifted back up (0.0674, 0.0683, 0.0665…) while the training loss kept falling to 0.0035. `load_best_model_at_end` therefore kept checkpoint-700 rather than the final checkpoint-1014. Textbook mild overfitting, caught only because the split is now wired into the trainer; before this change no validation ran at all.

9.8 The model can only learn what its lexicons contain

A worked example. `tgk` resolves correctly because SLANG contains it and 102 training rows say *tengok*. `aq` failed before the merge because it is in BM_dict but not SLANG, so all 25 of its training rows said “leave it alone”. `tdr` fails still: it is in neither lexicon and appears **zero** times in the data, so there is nothing to learn from and no amount of retraining will fix it.

Of the 132 human-corrected gold tokens, 28 are in SLANG, 37 in BM_dict only, and **67 in neither**. Those 67 are reachable only by generalisation. The fine-tune is therefore best described as distilling the lexicons into

the weights, plus whatever it generalises on top — and the honest subset is what measures the second part.

9.9 The earlier adapter was trained on differently labelled targets

The existing lemmatizer_lora_yt was trained on a 2,640-row file whose targets had been rewritten after the fact by a script that has since been deleted (recoverable with `git show 48b6697:manual_rework_data.py`). It read the already-lemmatised silver target and lemmatised it again, so 1,993 of 2,640 targets differed from the silver labels, while the validation file did not — the two halves were labelled by different systems. The new 2,700/300 split uses silver labels throughout, so train and dev finally share one labeller. Retrain before reporting.

10 What was removed, and what remains

The repository previously held three pipelines. Everything belonging to the other two has been deleted, freeing 248 MB. All of it is tracked in git and recoverable with `git checkout -- <path>`.

10.1 Removed

Path	Size	Belonged to
unsloth/lemmatizer_lora/	110 MB	the first seed500 adapter
unsloth/lemmatizer_lora_yt8k/	111 MB	the --n 8000 adapter
unsloth/data/training_8k.jsonl, val_8k.jsonl	10.7 MB	the yt8k adapter
data/silver_8k.jsonl	6.7 MB	the yt8k adapter
data/model_predictions_lemmatizer_lora_yt8k.json	12 KB	the yt8k adapter
unsloth/data/train_linked.jsonl, val_linked*.jsonl	7.3 MB	the v2 reasoning adapter
malay_lemmatisation_{alpaca,chat,text}.jsonl	2.4 MB	the v2 source registry
unsloth/data/val_seed.jsonl	69 KB	the seed500 run
unsloth/review_queue.tsv	29 KB	the v2 annotation queue

10.2 Three code changes the removals forced

Only the minimum needed to stop the deletions leaving broken commands. No behaviour beyond these was touched.

File	Before	After
unsloth/chat.py	default adapter `lemmatizer_lora` (deleted)	`lemmatizer_lora_yt`
crawl/evaluate.py	default adapter `lemmatizer_lora` (deleted)	`lemmatizer_lora_yt`
unsloth/llm.py	`--eval` read the deleted val_seed.jsonl	reads data/dev_300_annotated.jsonl, the dev split with its labels kept

So `python chat.py` and `python crawl/evaluate.py` now need no `--adapter` flag at all: the only adapter left is the right one.

10.3 What remains, and why

Path	Role	Read at inference?
unsloth/chat.py	the command	yes — the entry point
unsloth/skill.py	v1 prompt, the nine rule tags, rule derivation	yes
unsloth/preprocess.py	the four cleaning stages	yes
unsloth/databuilder.py	SLANG 136, PARTICLES 31, ROOTS 434, NEVER_STRIP 101, EN_HINT 49	yes
malindo_morph/BM_dict.csv	590 rojak → standard pairs, 521 of them absent from SLANG	no — labelling only
unsloth/sentidata/	6 files → 1,626 en / 123 ms vocabulary	yes

Path	Role	Read at inference?
malindo_morph/malindo_dic_2024.tsv	255,941-row morphological lexicon	yes
unsloth/lemmatizer_lora_yt/	the adapter and its tokenizer, 114.6 MB	yes
unsloth/llm.py	training	no — produced the adapter
crawl/youtube_crawl.py	collection	no
crawl/build_corpus.py	the five-layer filter	no
crawl/make_goldsheet.py	the gold test set	no
crawl/build_silver.py	labelling and the 90/10 split	no
crawl/evaluate.py	evaluation on gold	no
crawl/verify_splits.py	asserts the three-way design	no

Also kept at your request and untouched: app.py, main.py, Preprocess/, malindo_morph/dictionary.py, unsloth/{gpu,test_model,sample_sources}.py, crawl/score_filter.py and the notebook. None is reachable from this pipeline, so none affects it.

11 Report mapping and citations

11.1 Which material goes where

Report section	What to put there	From this document
Data collection	Source, method, period, channels, volume, ethics	Section 3
Data preprocessing	The five-layer filter, sentence splitting, banding	Section 4
Annotation / labelling	Silver labelling and the gold annotation	Sections 4.1, 8.1
Experimental setup	The 2,700 / 300 / 150 three-way split	Section 5
Model / training	QLoRA, loss masking, hyperparameters	Sections 6, 7
System description	What runs at inference and in what order	Sections 1, 10.3
Evaluation	Metrics, baselines, and the results table	Section 8
Limitations	Every item in section 9	Section 9

11.2 What to cite

Resource	Cite as	Size used
MALINDO Morph	MALINDO_Morph, 2024 release. github.com/matbahasa/MALINDO_Morph	255,941 rows
Llama 3.2 3B	Meta Llama 3.2 3B Instruct, 4-bit via Unsloth	3B params
Unsloth	Unsloth: QLoRA fine-tuning framework	—
TRL	TRL: Transformer Reinforcement Learning (SFTTrainer)	—
YouTube data	YouTube Data API v3, collected by the author	21,291 comments
SLANG / PARTICLES	Compiled by the author	136 / 31
Gold annotation	Annotated by the author	150 sentences

11.3 Phrasing that keeps the distinctions straight

- “The training set was *constructed using* MALINDO and the SLANG lexicon” — not “the model was *trained on*” them.
- “3,000 sentences were *sampled* from the 10,099 that passed the filter” — not “filtered down to 3,000”.
- “Labels were generated by rule and were not human-verified” — the 150-sentence gold set is the only human-verified data, and it is held out.
- “The 90/10 split is a fixed design choice” — not an optimum. The experiment tests fine-tuning against the base model and the rule baselines, not the ratio.
- “Fine-tuning” and “training” describe the same single step; there is no separate weight-update stage.

11.4 The commands, in order

```
# 1a. relabel and re-split from the corpus (after a lexicon change)
python crawl/build_silver.py --n 3000

# 1b. or re-split only, keeping the existing labels
python crawl/build_silver.py --resplit

# 2. assert the three-way design holds
```

```
python crawl/verify_splits.py

# 3. train  (USE_REASONING=0 pins the v1 prompt)
cd unsloth
USE_REASONING=0 ADAPTER_DIR=lemmatizer_lora_yt python llm.py --train

# 4. evaluate everything on the gold test set, once
cd ..
python crawl/evaluate.py --model --base

# 5. re-score the cache later without a GPU
python crawl/evaluate.py --reuse --base

# 6. use the model
cd unsloth
python chat.py --adapter lemmatizer_lora_yt
```

Companion file

lemmatizer_lora_yt_pipeline_data.xlsx — every record behind this document: the raw crawl, the corpus, the gold and silver sets token by token, the training rows, the lexicons, and the model's predictions on all 150 gold sentences. Note that it was generated before this cleanup and still describes the 2,640 / 360 split; regenerate it after retraining if you need the two to agree.

12 Results

Every figure in this chapter was produced by one command against the 150-sentence gold test set, and can be reproduced:

```
python crawl/evaluate.py --model --base    # one GPU pass
python crawl/evaluate.py --reuse           # re-score, no GPU
```

The headline finding, stated up front

Fine-tuning worked. The same 3B model went from **12.9%** token accuracy to **93.8%**, from 10.9% F1 to 90.0%, and from failing to produce a token-aligned sentence 90% of the time to 6.7%. Almost all of that is the 24.3 M-parameter adapter.

The rule baseline still edges it. 95.1% against 93.8%, 92.0% F1 against 90.0%. On the full test set that gap is statistically significant (McNemar p = 3.4e-05). On the 132 tokens a human actually corrected — the only part not graded by the rule system itself — it is **not** significant (p = 0.065).

Both statements belong in the report. The second does not cancel the first, and the first does not excuse the second.

12.1 Overall performance

What each measure means, in plain words

Every measure below answers a different question about the same 150 sentences. None of them is the “real” one; each is blind to something the next one catches.

Measure	The question it answers	Why it is not enough on its own
Token accuracy	Out of all 1,786 words, how many did the system get right?	Two thirds of Malay words are already their own lemma, so a system that changes nothing still scores 66.7%
CHANGED accuracy	Of the 595 words that actually needed fixing, how many were fixed correctly?	Says nothing about words wrongly changed that should have been left alone
Unchanged accuracy	Of the 1,191 words that were already correct, how many were left alone?	A system that never changes anything scores a perfect 100% here
Precision	When the system decided to change a word, how often was that change right?	A very cautious system can be near-perfect here by rarely acting at all
Recall	Of all the words needing a change, how many did it change correctly? **Same number as CHANGED.**	A reckless system can score well by changing everything
F1	One number balancing precision and recall — being good at both at once.	Being a single number, it hides which of the two is the weak half
Sentence exact match	How many whole sentences came out perfect, every single word right?	The harshest measure: one wrong word in thirty fails the whole sentence
WER / CER	How many word (or character) edits would turn the answer into the correct sentence?	Lower is better and 0 is perfect, but the number has no natural ceiling
Format-failure rate	How often did the model return the wrong *number* of words, so its answer could not be lined up at all?	Applies only to the generative systems; the lexicon ones cannot fail this way

Why accuracy alone would mislead

Imagine a medical test for a disease that only 3 people in 100 have. A test that answers "healthy" to everybody is **97% accurate** and completely useless.

Lemmatisation has the same shape. 1,191 of 1,786 words are already their own lemma, so answering "no change" every time scores 66.7%. That is why the `copy` row exists, and why CHANGED, precision and F1 are reported next to accuracy rather than instead of it.

The results

All six systems on the same held-out test set: 150 human-verified sentences, 1,786 tokens, of which 595 (33%) require a change and 1,191 are already their own lemma.

System	Accuracy	Precision	Recall (= CHANGED)	F1	Sent-exact	Format-fail
Copy (floor)	66.7%	0.0%	0.0%	0.0%	4.0%	—
MALINDO lemma	66.9%	17.9%	0.8%	1.6%	4.7%	—
MALINDO root	74.2%	97.8%	22.5%	36.6%	6.7%	—
Rules (full)	**95.1%**	96.0%	88.4%	**92.0%**	**64.7%**	—
Base Llama 3.2 3B	12.9%	22.4%	7.2%	10.9%	0.7%	90.0%
Fine-tuned (LoRA)	**93.8%**	94.8%	85.7%	**90.0%**	**58.0%**	6.7%

Table 4.1 — every headline figure in one place. *Recall* and *CHANGED* are the same measurement under two names, shown once. The confusion-matrix counts behind precision and recall are in section 12.2, and the unchanged-accuracy column is there too — it is left out here because `copy` scores 100% on it, which tells the reader nothing.

Read the base-model row carefully. 12.9% is *below* the do-nothing floor of 66.7%. An un-fine-tuned Llama 3.2 3B given the v1 prompt does not lemmatise badly; it does something else entirely — explains, translates, answers — and its 90% format-failure rate is the honest description of what happened. The 12.9% is generous: it is scored after LCS alignment rescued whatever could be rescued, rather than scoring a malformed answer as zero.

Why the three weak baselines are in the table

They look like padding. They are the opposite: without them, none of the other numbers can be interpreted. Each one isolates a single question.

Baseline	What it does	Its score	The question it settles
Copy	Nothing at all — returns every word exactly as typed	66.7%	**What do you get for free?** This is the floor. Against it, the fine-tune's 93.8% is worth 27 points of real work, not 94. Any system scoring below 66.7% — the base model does — is worse than useless
MALINDO lemma	Looks each word up in the dictionary and takes its lemma column	66.9%	**Why not just use an existing Malay dictionary?** Because it is worth **0.2 points** over doing nothing. Standard dictionaries have no entry for `nk`, `tdr` or `sngt`. This single row justifies the whole project
MALINDO root	The same dictionary, but reading the *root* column instead	74.2%	**Is it the resource or how you use it?** The same file, queried differently, is worth **+7.3 points**. That is a free gain from a design decision, and it shows the dictionary is not the bottleneck — the query was

Together they make a ladder, and each rung adds exactly one thing: **66.7%** do nothing → **66.9%** add a dictionary → **74.2%** query it better → **95.1%** add slang, affix and spelling rules → **93.8%** replace the rules with a fine-tuned model. A reader can see where every point came from. Delete the first three rows and the last two become unreadable numbers with nothing to stand against — and the first question in a viva, *"why not just use a dictionary?"*, has no answer.

12.2 Precision, recall and F1

Lemmatisation framed as the decision to **change** a token, which is what makes over-correction visible. Accuracy hides it; precision does not.

System	TP	FP	FN	TN	Precision	Recall	F1
copy	0	0	595	1,191	0.0%	0.0%	0.0%
malindo	5	23	590	1,189	17.9%	0.8%	1.6%
dict-root	134	3	461	1,191	97.8%	22.5%	36.6%
rules	526	22	69	1,173	96.0%	88.4%	**92.0%**
base-llama	43	149	552	1,094	22.4%	7.2%	10.9%
fine-tuned	510	28	85	1,172	94.8%	85.7%	**90.0%**

Table 4.2 — the counts behind the percentages in Table 4.1, plus the unchanged-accuracy column (TN). Recall is the CHANGED column under another name, so the two are never independent evidence.

- **The base model over-corrects catastrophically**: 149 false positives against 43 true positives. It rewrites tokens that were already correct more than three times as often as it fixes one.
- **The fine-tune is nearly as precise as the rules** (94.8% vs 96.0%) and slightly less willing to act (85.7% vs 88.4% recall). It is the same kind of system, marginally more cautious — not a different trade-off.
- **dict-root shows why precision earns its place**: 97.8% precision on 22.5% recall. It is almost always right when it acts, and it hardly ever acts. Accuracy alone cannot express that.

12.3 Sequence-level error

System	WER	CER
copy	0.3396	0.1699
malindo	0.3380	0.1750
dict-root	0.2653	0.1284
rules	**0.0638**	**0.0244**
base-llama	2.1115	2.2286
fine-tuned	**0.0660**	**0.0322**

Table 4.3 — corpus-level edit distance against the gold lemma sentence. For the two neural systems the hypothesis is the **raw generation**, not the aligned tokens, so words the alignment absorbed still count — otherwise WER would forgive exactly the format failures it exists to catch.

A WER above 2.0 is not a rounding error. The base model emitted more than twice as many wrong words as there were words in the reference. That is the numerical signature of a model that answered the question instead of performing the task, and it is the clearest single argument in this report that fine-tuning was necessary rather than merely helpful. The fine-tune brings it to 0.0660 — a **32-fold** reduction.

12.4 The honest subset

Why this table, and not Table 4.1, decides it

The gold sheet was **pre-filled by the rule system** and then corrected by hand. The annotator accepted the rule output on 1,654 of 1,786 tokens and overrode it on 132. On those 1,654, `rules` is being graded against its own suggestions, and scores accordingly.

The 132 overridden tokens are the only part of the test set independent of the rule system. They are where a fine-tune can show it adds something a lexicon cannot — and where a rule baseline cannot mark its own homework.

System	Honest subset (n = 132)	Full set	Gap
copy	3.0%	66.7%	–63.7
malindo	1.5%	66.9%	–65.4
dict-root	3.0%	74.2%	–71.2
rules	**47.0%**	95.1%	–48.1
base-llama	0.8%	12.9%	–12.1
fine-tuned	**41.7%**	93.8%	–46.9

Table 4.4 — the difference between the two accuracy columns is the size of the circularity. Every system loses 45–70 points when the rule system stops grading itself.

5.3 points separate the two contenders here, and at n = 132 that is inside the noise (McNemar p = 0.065, section 12.5). The honest statement is that the fine-tuned model and the rule cascade are **statistically indistinguishable** on the only tokens that can tell them apart. Both are also failing more than half of them, which is the more important finding: 58% and 53% of genuinely hard rojak defeats both systems.

12.5 Statistical significance

Both systems are scored on the *same* tokens, so the outcomes are paired and only the disagreements carry information. **McNemar's exact test** is the correct one, reported here against `rules`.

System	Subset	It wins	rules wins	p	Verdict
copy	1,786 tokens	18	526	7.4e-131	significant
copy	132 honest	2	60	8.5e-16	significant
malindo	1,786 tokens	16	521	8.4e-132	significant
malindo	132 honest	0	60	1.7e-18	significant
dict-root	1,786 tokens	19	393	5.2e-92	significant
dict-root	132 honest	2	60	8.5e-16	significant
fine-tuned	1,786 tokens	4	27	3.4e-05	**significant**
fine-tuned	132 honest	2	9	0.065	**NOT significant**

Table 4.5 — a small p says the gap is unlikely to be chance. It does not say the gap is large.

The two fine-tuned rows are the whole result in miniature. Across all 1,786 tokens the systems disagree on only 31, and rules wins 27 of them — consistent enough to be significant, but a gap of **31 tokens out of 1,786**. Restricted to the tokens a human corrected, it is 2 against 9, and 9-versus-2 is what a fair coin does roughly once in fifteen tries.

12.6 Head-to-head

The three systems the project exists to compare, on every measure, in one place. `Copy` is the floor: it returns every token unchanged, so anything failing to beat it is doing worse than nothing.

Measure	Copy (floor)	Rule-based	Base 3B (before)	Fine-tuned (after)
Token accuracy	66.7%	**95.1%**	12.9%	93.8%
CHANGED accuracy	0.0%	**88.4%**	7.2%	85.7%
Precision	0.0%	**96.0%**	22.4%	94.8%
Recall	0.0%	**88.4%**	7.2%	85.7%
F1	0.0%	**92.0%**	10.9%	90.0%
Sentence exact match	4.0%	**64.7%**	0.7%	58.0%
WER	0.3396	**0.0638**	2.1115	0.0660
CER	0.1699	**0.0244**	2.2286	0.0322
Format-failure rate	n/a	n/a	90.0%	6.7%
Honest subset (132)	3.0%	**47.0%**	0.8%	41.7%

Table 4.6 — rule-based leads every row. The base column isolates what the prompt alone achieves with no task-specific training; base → fine-tuned is the contribution of the adapter, and fine-tuned vs rule-based is whether the neural approach was worth taking.

Two different questions live in this table and the report should answer both explicitly. **Did fine-tuning work?** Base versus fine-tuned: emphatically yes, on every measure, by margins between 47 and 83 points. **Did it beat the rule system?** No — not on this test set. Both answers are legitimate findings.

12.7 Error analysis: where the 31 disagreements are

The aggregate gap is small enough to inspect token by token, which is the only way to know whether it is a systematic weakness or a handful of accidents. Everything in this section is reproduced by one command, which re-derives it from the gold file and the cached predictions using the same alignment the scores above were computed from — so the table cannot drift from the numbers:

```
python crawl/evaluate.py --reuse --errors
```

Outcome	Tokens	Reading
Only the fine-tune correct	4	the model's entire advantage over the rule cascade
Only rules correct	27	over-eager affix stripping in the model, mostly
Both wrong	83	**the real headroom** — shared blind spots, 2.7× the size of the gap between the systems
Both correct	1,672	the overwhelming majority

Every token the fine-tune won, in full — there are only four, and a report that lists all four is more credible than one that quotes the best:

Token	Gold	Rules said	Why rules missed it
tolonglahh	tolong	tolonglahh	elongation and the *-lah* particle stacked; the cascade handles either alone
y	y	**yang**	"umur 42 **y**" — *years*, not *yang*. **Context.**
taknak	nak	**tak nak**	the cascade *did* split the fused negation, into two words; gold keeps only *nak*. **Arguably a gold-label question** — dropping *tak* loses the negation
endingnya	ending	endingnya	an English stem carrying a Malay *-nya* suffix

On the temptation to build the report around `y`

`y` → `yang` is a genuine context failure: `propose()` takes one token and cannot see that a number precedes it. The model can. It is the most interesting example in the whole evaluation.

It is also **one token**. Counted across the test set, exactly **one surface form out of 934** requires two different lemmas in two different sentences — and that one (`tau` → `tahu / tau`, 3 occurrences, 0.2% of tokens) looks like an annotation inconsistency rather than real ambiguity.

So the honest form of the claim is: *context-dependence is a structural limitation of any per-token rule system, and this corpus contains almost none of it.* Quote `y` as an illustration under a counted headline. Do not build an argument on n = 1 — the first question in a viva will be how many such cases there were.

The 27 losses are two different failures and should not be reported as one. **Seven of them are alignment failures**, not wrong lemmas: the model returned a sentence whose word count did not match the input, so five tokens of one sentence (`tuan peguam tu`…) and two others came back with no lemma at all. Those belong to the 6.7% format-failure rate already reported in Table 4.1, and counting them twice would overstate the lemmatisation gap.

The remaining **20 are genuine wrong answers**, and they have a clear pattern worth naming: **over-eager affix stripping**. `manaikkan` → `manaik` (gold *naik*), `selamanya` → `selama` (gold *lama*), `pengantin` → `kantin` (gold *pengantin*) — it has learned that prefixes come off and applies it one step too far, including to words where the prefix is part of the stem. Two tokens produced no output at all. This is the concrete direction for future work: the failures are systematic, not random, so more affix-boundary examples in training should move them.

Why the rule system is wrong when it is wrong

The comparison so far says the rule cascade scores higher. This subsection says **where it breaks**, because "95.1% accurate" and "structurally limited" are both true and the report needs the second one evidenced rather than asserted.

Run `propose()` over the gold set and it produces a wrong lemma for **87 of 1,786 tokens**. Seventy of those 87 fall on the 132-token honest subset — that is what 47.0% accuracy means in practice. Grouping them by the branch of the cascade that produced the wrong answer turns the score into a diagnosis:

Rule branch	Errors on the honest subset	What went wrong	Example
oov	41	**Nothing in the cascade matched.** No particle rule, no slang entry, no BM_dict entry, no MALINDO row — so the token is returned unchanged. The cascade has no mechanism for guessing.	`skrng` → *sekarang*, `prcya` → *percaya*, `tgkp` → *tangkap*, `pnyanyi` → *nyanyi* — all left as typed
dict_check	17	**MALINDO contains the informal spelling itself**, so the lookup succeeds, the cascade stops, and the wrong answer is returned with confidence. A miss would have been safer than a false hit.	`tdk` → *tidak* (×3), `n` → *dan* (×4), `pon` → *pun* (×3), `dri` → *dari*
dict	3	**The dictionary made it worse.** A correct root was mapped to a more inflected form — a lemmatiser running backwards.	`tangkap` → rules answered **menangkap**, gold is *tangkap*
ambiguous	3	Several MALINDO candidates; the cascade takes the first. Dialect forms lose here.	`depa` (Kedah for *they*) → gold *mereka*, rules left it unchanged
affix	1	**Stripping changed the meaning.** The negation was removed along with the prefix.	`taktau` → rules answered **tau**, gold is *tidak tahu* — the opposite
elong, particle, slang	5	The right branch fired but stopped one step short of the lemma.	`kelakooo` → **kelako**, gold *kelakar*; `dlu` → **dahulu**, gold *dulu*

Table 4.7 — reproduce with `python crawl/evaluate.py --reuse --errors` and the rule tags printed by `python crawl/try_rules.py`.

What these categories actually show

- **The dominant failure is coverage, not logic.** 41 of 70 — 59% — are `oov`: the word was in no lexicon, so nothing happened. Rojak is an open vocabulary in which vowel-dropped spellings are generated freely (`skrng`, `panjangkn`, `dprmudahkan`), and a lookup table can only ever contain the ones somebody has already typed in.
- **A lexicon that is wrong is worse than one that is missing.** The 17 `dict_check` errors are cases where MALINDO held the informal form, so the cascade stopped early and returned it. `tdk` stays `tdk` not because the system could not resolve it but because it believed it already had.
- **Rules can invert meaning.** `taktau` → `tau` turns "doesn't know" into "knows". A deterministic system will make that error identically every time it sees the form.
- **Every one of these is fixed by a human hand-editing a file.** That is the maintenance argument in concrete form: 70 corrections means 70 lexicon edits, each of which can itself be wrong — as four BM_dict entries in this project were.

The honest half of this argument

The fine-tuned model does not currently fix these either. On most of the 41 `oov` tokens it returns the same unchanged word the cascade did — which is exactly what the 83 both-wrong tokens are. Of the 70, the model rescues 2.

That is not a coincidence, and it is the most important sentence in this chapter: **the model was trained on labels the cascade produced, and on these tokens the cascade's label was "leave it alone".** The model learned the rule system's blind spot along with its knowledge.

So the correct claim is *architectural*, not empirical: a sequence model **can** learn that `skrng` → `sekarang` from examples, because vowel-dropping is a regular pattern; a lookup table cannot, however many examples it sees. Realising that capability needs human-labelled training data, which is why it is the first item of future work in 15.6.

12.8 System selection: why the fine-tuned model is deployed

Section 12.6 shows rule-based ahead on every row. This section states plainly why the fine-tuned model is nonetheless the system the interface presents, and what that claim does **not** rest on.

Against the base model: no contest

This half needs no qualification. The base model is worse than returning the input unchanged (12.9% against a 66.7% floor), over-corrects three times more often than it corrects, fails to produce an alignable sentence 90% of the time, and has a WER above 2.0. McNemar rejects it against every other system at p < 1e-15. It cannot be deployed and it is not a close call.

Against the rule cascade: the honest case

Here the accuracy argument runs the other way, and the report must say so. The case for deploying the fine-tune rests on four things, none of which is a claim to higher test-set accuracy:

#	Argument	Evidence	Strength
1	On the only non-circular measure they are indistinguishable	Honest subset 47.0% vs 41.7%, McNemar **p = 0.065**. The 95.1% headline for rules comes from a test set it pre-filled; it grades itself on 1,654 of 1,786 tokens.	**Strong** — measured
2	The gap on the full set is 31 tokens	4 wins against 27 losses out of 1,786. Statistically real (p = 3.4e-05) but a 1.3-point difference in accuracy.	**Strong** — measured
3	Open vocabulary	The cascade is a closed lookup: a token in no lexicon returns `oov` unchanged. The model emits a lemma for any input, including spellings never seen.	**Structural, not measured here** — gold contains no out-of-lexicon tokens needing change
4	Maintenance path	Extending the cascade means hand-editing SLANG and BM_dict per word. Extending the model means labelling more sentences and retraining — which is also how the cascade's own gaps get found.	**Engineering judgement**

This deviates from the rule fixed in 12.7 of the earlier draft - say so, do not hide it

The selection criteria were written down **before** these numbers existed, and they said: if rule-based is ahead or the two tie, the GUI defaults to rule-based. Taken literally, the measurement points at the rule cascade — it wins criterion 2 (F1), criterion 3 (format failure: 0% against 6.7%) and criterion 4 (no GPU).

Deploying the fine-tune anyway is defensible, but only as a **disclosed deviation with its reasons stated**, which is what this section is. A report that quietly swaps the winner after seeing the data has thrown away the reason for pre-registering anything. A report that says *"the criteria favoured rules; we deployed the model for reasons 1–4 and kept rules as a live fallback"* is doing science.

Never write that the fine-tuned model is more accurate. It is not, on this test set, and the number is one table away.

What is actually deployed

The fine-tuned model is the system. It produces every lemma the system outputs. The rule cascade is retained in the interface as a **comparison and diagnostic view**, not as a second opinion that is merged into the answer — the two predictions are never combined, and describing the result as a hybrid would overstate the integration.

- **The model produces the lemma.** Every word the system outputs comes from the adapter. No rule is consulted before generation and no rule output is merged into it afterwards.
- **The rule cascade is a parallel view.** It runs on the CPU for every sentence and is displayed alongside, with the per-token tag that fired. Its answer is never substituted into the model's — a viewer compares them, the system does not.
- **The one point of contact is presentational.** When the model returns a different number of words than there were input tokens, `align_to_tokens()` uses the rule sequence as a reference to decide which model word belongs on which table row. It changes no lemma; unmatched rows show a dash rather than the rule answer.
- **Nothing is hidden.** Showing the alternatives is the honest representation of a result where the two are 5 points and one non-significant p-value apart.

12.9 Discussion against the project objectives

Objective	Evidence	Status
Build a corpus of authentic Malay rojak text	21,291 comments from 9 Malaysian channels; a three-layer filter retained 10,099 rojak sentences, banded by difficulty. Author identifiers discarded at collection.	**Achieved**
Produce a labelled dataset for supervised fine-tuning	8,000 sentences labelled by a rule cascade; 96,552 tokens, 64.3% carrying a label the system treats as reliable.	**Achieved**
Establish an independent test set	150 sentences hand-corrected, 132 tokens overridden. Excluded from training by identifier, asserted rather than assumed, verified by an independent script.	**Achieved**
Fine-tune an LLM for rojak lemmatisation	QLoRA adapter over Llama 3.2 3B, 24.3M trainable parameters (0.76% of the base), checkpoint selected on a held-out dev split.	**Achieved**
Show fine-tuning improves on the alternatives	Against the **base model**: yes, decisively — +80.9 points accuracy, +79.1 F1, format failure 90.0% → 6.7%, $p < 1e\text{-}15$. Against the **rule cascade**: no — 93.8% against 95.1%, and indistinguishable on the honest subset ($p = 0.065$).	**Partly achieved**

Table 4.8 — "partly achieved" is the accurate entry and it costs nothing to write. The objective as phrased contains two claims; one is met overwhelmingly and one is not met on this test set.

Why the fine-tune could not have won, by construction

The most defensible reading of the whole comparison, and the one that shows command of the pipeline: **the model was trained on targets written by the rule cascade.** 8,000 silver sentences, every label produced by `propose()`. The fine-tune is therefore **distilling** the rule system, and a student cannot systematically exceed a teacher that wrote every answer in its textbook.

Reaching 93.8% of a 95.1% teacher — 98.6% of its performance — through a 24.3 M-parameter adapter is a successful distillation, and it is the correct frame for this result. It also identifies the single change most likely to reverse the ranking: **train on human labels, not rule labels.** The 132-token honest subset is the seed of exactly that dataset, and both systems failing over half of it (section 12.4) shows the headroom is there.

Threats to validity

- **n = 132 on the deciding measure.** The 95% interval is roughly ±8.5 points, so gaps under ~12 points cannot be resolved. The observed gap is 5.3. More gold sentences is the highest-value remaining work.
- **The test set is rule-pre-filled.** Corrected by hand, but the starting point biases every non-honest-subset number toward `rules`. Stated wherever those numbers appear.
- **Single training run, single seed** (3407). No variance estimate across restarts, so a 1.3-point accuracy difference could partly be run-to-run noise.
- **One annotator.** No inter-annotator agreement figure, so gold label consistency is unquantified — the `tau` → `tahu / tau` inconsistency found in 12.7 is direct evidence that it matters.

12.11 Algorithm analysis

Complexity of each component, with the measured cost beside it. *c* is characters in a sentence, *n* tokens, *m* generated words, *T* total tokens in the test set, *R* rows in the MALINDO file, *S* optimiser steps, *B* batch size and *L* sequence length.

Component	Time	Space	Measured
Preprocess (4 stages)	O(*c*)	O(*c*)	31.7 µs / sentence
propose() — rule labelling	O(1) expected, hash lookups	O(V) lexicons	1.3 µs / token
derive_rule() — rule tagging	O(1) expected	O(V)	31.8 µs / token
align_strict()	O(*n*)	O(*n*)	negligible
align_lcs() (fallback)	O(*m* × *n*) worst case	O(*m* × *n*)	fires on format failures only
edit_distance() for WER / CER	O(|a| × |b|)	O(|b|)	once per sentence per system
score()	O(*T*)	O(*T*)	1,786 tokens
MALINDO load (one-off)	O(*R*), R = 255,941	174,168 surface keys	**2.25 s**, once per process
sentidata induction (one-off)	O(lines)	1,749 vocabulary items	0.112 s, once per process
QLoRA fine-tuning	O(*S* × *B* × *L*²) attention-dominated	8 GB VRAM at 4-bit	2,700 steps, 3 epochs
Generation (greedy)	O(*G* × *L*) autoregressive	base + 97.3 MB adapter	150 sentences per eval pass

Three observations worth making in the write-up:

- **The rule path is dominated by a one-off cost, not per-token work.** Labelling a token takes 1.3 µs, but loading MALINDO takes 2.25 s — the equivalent of about 1.7 million tokens. This is why predictions are cached per system and why the dictionary is loaded once per process rather than per call.
- **The only super-linear component is the LCS fallback**, and it runs only when the model breaks format. Strict positional alignment is linear and handles the well-formed majority, so the quadratic path is paid exactly where it buys something.
- **Fine-tuning cost is set by the adapter, not the model.** Only 24.3M of roughly 3.2B parameters receive gradient (0.76%), and the base is frozen in 4-bit — which is the whole reason the run fits in 8 GB of consumer VRAM.

12.12 Coding

Python throughout. The pipeline is 11 modules and just under 5,000 lines, of which about 10% are explanatory comments — a deliberate ratio, because several components encode decisions that are wrong-looking until the reason is stated.

Module	Lines	Role
unsloth/skill.py	976	system prompt, nine-tag rule vocabulary, rule derivation, alignment
crawl/evaluate.py	641	six systems, two alignment strategies, all metrics
crawl/build_corpus.py	545	the five-layer rojak filter and difficulty banding
unsloth/databuilder.py	526	SLANG, PARTICLES, ROOTS, NEVER_STRIP, EN_HINT lexicons
unsloth/llm.py	454	the three training stages
crawl/make_goldsheet.py	447	gold sheet construction and the rule cascade
crawl/youtube_crawl.py	360	YouTube Data API v3 collection with quota accounting
crawl/build_silver.py	315	silver labelling and the stratified split
unsloth/preprocess.py	284	the four cleaning stages
unsloth/chat.py	278	the interactive entry point
crawl/verify_splits.py	128	21 assertions over the split files
Total	**4,954**	3,671 code, 505 comment, 98 docstrings

Dependencies

Layer	Packages	Needed for
Collection	requests	the YouTube Data API
Labelling	pandas, pyenchant	MALINDO parsing, English detection
Training	unsloth, trl, transformers, peft, datasets, torch, bitsandbytes	QLoRA fine-tuning; GPU only
Inference	unsloth, transformers, torch	loading base + adapter
No dependencies	preprocess.py, skill.py core, verify_splits.py	stdlib only, so they stay testable without a GPU

Design decisions worth defending in a viva

- **One source of truth for the system prompt.** It is read from the adapter, never retyped. A stale copy in chat.py once asked for an output format the model had never been trained on, and every answer came out nonsense; importing it from llm.py replaced that with a subtler version of the same bug, because llm.py chooses the prompt from an environment variable describing a *training* run.
- **Assertions, not assumptions.** The build refuses to run if it cannot find the gold identifiers to exclude, and asserts the exclusion afterwards; verify_splits.py re-derives all 21 properties from the files themselves rather than trusting the manifest's own bookkeeping.
- **Caches keyed by system.** Model predictions are stored under a filename derived from the adapter, because a shared cache once let predictions from one model be scored as another's.
- **Version-tolerant configuration.** The trainer is constructed through a helper that asks the installed TRL and transformers what arguments they accept and prints anything it drops — a silently ignored eval_strategy is how a project ends up believing it has validation when it has none.

- **Components runnable in isolation.** `preprocess.py --demo`, `skill.py --derive`, `skill.py --score` and `evaluate.py --reuse` all run without a GPU, which is what makes the non-neural half of the system testable at all.

13 Graphical user interface

The demonstrator is a single-page **Streamlit** application, `app.py`, at the repository root. It exists to make the comparison in section 12 visible on a sentence the user types, rather than only in a table of aggregate scores.

13.1 Framework and how it runs

Item	Detail
Framework	Streamlit — a Python library that renders a script top-to-bottom as a web page and re-runs the whole script on every interaction
Entry point	`app.py` (263 lines), repository root
Command	`streamlit run app.py`
Served at	`http://localhost:8501` in a browser
Hard dependencies	streamlit, and the project's own `crawl/make_goldsheet.py` plus the MALINDO Morph CSV
Optional dependencies	unsloth, torch, transformers, bitsandbytes and a CUDA GPU — needed only if the fine-tuned column is switched on

Why Streamlit rather than Flask or Django: the deliverable is a comparison table and three coloured output boxes, with no accounts, no database and no persisted state. Streamlit renders that from ordinary Python with no HTML, no routing and no front-end build step, and its `@st.cache_resource` decorator solves the one hard problem the page has — keeping a 255,941-row lexicon and a 3B-parameter model in memory across re-runs.

Why the file is not called streamlit.py

A module named `streamlit.py` in the project root shadows the installed `streamlit` package, so `import streamlit` finds the project's own file and the application cannot start.

The same class of collision is handled again inside `load_model()`, where the project's `unsloth/` folder shadows the `unsloth` library: the loader temporarily removes the repository root from `sys.path`, drops the cached module entry, imports the real library, and restores the path in a `finally` block.

13.2 What the page shows

Three systems run on the same sentence and are displayed together, because the result this project reports is a difference between systems, not one system's output.

Column	What it runs	Cost	Always on?
MALINDO	Dictionary lookup only — `pick(lex.get(token))`. No slang lexicon, no orthographic repair. This is what an off-the-shelf Malay lemmatiser gives you.	CPU	yes
Rules	`propose()` — the full labelling cascade: particles, slang lexicon, BM_dict, reduplication, elongation, MALINDO root, MALINDO lemma, English detection.	CPU	yes
Fine-tune	The QLoRA adapter, loaded through Unsloth in 4-bit and decoded greedily.	GPU	**no** — behind a checkbox, off by default

A fourth column, *rule applied*, carries the tag `propose()` returned (`slang`, `affix`, `redup`, `oov` and so on), so the rule path is explainable token by token. A fifth, *only the model got it*, flags rows where the fine-tune departed from both lexicon systems — those rows are the qualitative evidence for or against the model, and they are what a demonstration should be looking at.

13.3 Which model the GUI uses

Layer	Value
Base model	unsloth/Llama-3.2-3B-Instruct, loaded 4-bit (NF4)
Adapter	`unsloth/lemmatizer_lora_yt` — 24.3 M LoRA parameters, 97.3 MB
Selected by	a sidebar `selectbox` populated by globbing `unsloth/lemmatizer_lora*` for folders containing `adapter_model.safetensors`; `lemmatizer_lora_yt` is the default index
Prompt contract	**v1** — the model returns a bare lemma sentence, with no rule tags
Chat template	`llama-3.2`, applied via `get_chat_template()`
Decoding	`do_sample=False` (greedy), `max_new_tokens=200` — deterministic, so the same sentence gives the same answer every time it is demonstrated
Load time	roughly 30 s on first use, then cached for the life of the server process

Which system is the default

This follows from the selection rule fixed in section 12.7, and not from which component was the more interesting one to build. On every measure taken so far the rule path is ahead — 92.0% F1, 47.0% on the honest subset, 0.0244 CER, no format failures, CPU-only — and the neural columns of Table 4.6 are not yet measured. A page that defaulted to the model today would be defaulting to the system with **no** evidence behind it, which is why the checkbox ships off.

Once Table 4.6 is complete	What the GUI should do
Fine-tune beats rules by more than ~12 points on the honest subset (McNemar p < 0.05)	Default the checkbox to **on** where a GPU is present; keep the rule column visible as the comparison; fall back to the rule output automatically when alignment fails, so a format failure never reaches the user as a blank answer
The two overlap within the confidence interval, or rules is ahead	Keep the present behaviour — rules is the default answer and the model is an opt-in second opinion

The current architecture is correct under either outcome, which is why it was built this way: the rule path costs nothing and needs no GPU, so it can always run, and the model is an addition to the page rather than the page itself.

13.4 How a request flows through the page

#	Step	Code
1	User types a sentence and submits the form	`st.form` + `text_area` + `form_submit_button`
2	Guard: reject empty input, and input that leaves no word characters	`st.warning` then `st.stop()`
3	Lowercase and tokenise	`WORD_RE.findall(text.lower())`
4	MALINDO column	`run_malindo()`
5	Rules column, and the rule tag for each token	`run_rules()`, `rule_tags()`
6	*If enabled:* load the adapter (cached), generate, then align the sentence back to the input tokens	`load_model()`, `run_model()`, `align_to_tokens()`
7	Assemble one row per token and render the table	`st.dataframe`
8	Render the full sentences and the disagreement count	`st.columns(2)`, `st.info` / `st.success`

Step 6 runs **before** the table is drawn rather than being appended underneath it, so the model's answer can be a column alongside the other two. Appending it below as a footnote would have been easier and would have lost the comparison, which is the whole purpose of the page.

13.5 Four problems the page had to solve

1. The button that appeared to do nothing. Streamlit only commits a `text_area`'s value when the widget loses focus. Typing and then clicking in one motion sends the click before the text, so the first press did nothing and the user had to click twice. Wrapping both in an `st.form` submits them together, and gives Ctrl+Enter for free.

2. Aligning a sentence to a token list. The model returns a sentence, not a token list, and the counts drift — it expands `xleh` into `tidak boleh`, or spells a numeral out. When the counts agree the mapping is positional. When they differ, `align_to_tokens()` matches the model's words against the **rule pipeline's output** by longest common subsequence, not against the raw input tokens — both are lemma sequences and mostly agree, whereas matching against raw tokens would fail on every word the model correctly changed, which is most of them. A rule lemma can itself be several words, so each reference word records which token owns it. Anything left unmatched comes back as `None` and is displayed as a dash rather than guessed at.

3. Reloading a 255,941-row lexicon on every keystroke. Streamlit re-runs the entire script on every interaction. `@st.cache_resource` on `load_lexicons()` and `load_model()` makes both load once per process. The model cache is keyed on the adapter name, so switching adapters in the sidebar reloads and switching back does not.

4. Triton could not find its compiler. Windows Store Python reports a site-packages path that packages are not actually installed into, so Triton cannot locate the TinyCC it ships with and every kernel build fails. `_ensure_cc()` points `CC` at `triton/runtime/tcc/tcc.exe` when the variable is not already set, so the page works without the user having to remember an environment variable.

13.6 Failure handling

Failure	What the user sees
No adapter folder on disk	The sidebar says so; the two lexicon columns still work
No GPU, or the model raises during load or generation	`st.error` with the exception text, plus "the dictionary and rule columns are unaffected" — the model call sits in a `try/except` so it cannot take the page down with it
The model returns a word count that will not align	`st.warning` naming both counts, the unalignable cells shown as a dash, and the model's full raw sentence printed underneath so nothing is hidden
Input is empty, or is only digits and punctuation	A warning and an early `st.stop()`

The principle throughout is that **the optional component degrades, the page does not**. Every model failure leaves two working columns behind.

13.7 Defects found reviewing the GUI for this report

Three divergences between `app.py` and the rest of the pipeline, listed because a report that only describes what works is not a review.

Defect	Why it matters	Fix
The system prompt is retyped	`app.py` hardcodes its own copy of the v1 prompt string. It is byte-identical to `skill.SYSTEM_PROMPT_V1` today, so the GUI is correct *today* — but nothing keeps it that way, and the project's own rule is that the prompt is asked for, never retyped. Point the picker at a v2 adapter and the page would send a v1 prompt and silently get the wrong output format.	Call `skill.prompt_for(adapter_dir)`, as `chat.py` does

Defect	Why it matters	Fix
Input is not preprocessed	`chat.py` runs `preprocess()` — emoji stripping, elongation collapse, digit policy — before the model sees the text. `app.py` only lowercases and tokenises. The two front ends therefore feed the model differently, and a sentence can behave differently in the GUI than on the command line.	Share the preprocessing, or state the difference when demonstrating
HIDDEN_ADAPTERS is dead	It suppresses `lemmatizer_lora_yt8k`, which was deleted in the repository cleanup, so the set now excludes nothing while its comment still quotes scores from a model no longer on disk. The picker meanwhile now offers a second live adapter, `lemmatizer_lora_yt_newsplit`.	Delete the constant, or repoint it at whichever adapters should not be demonstrated

13.8 Scope of the interface

Stated plainly, so the report does not overclaim it:

- It is a **demonstrator**, not a deployed service. One sentence at a time, single user, local host, no authentication, no persistence, no batch mode and no API.
- It is **not part of the evaluation**. Every number in section 12 comes from `crawl/evaluate.py` over the 150-sentence gold test set. The GUI is qualitative evidence — useful for showing an examiner a specific token, worthless as a measurement, because sentences typed live are not a held-out sample of anything.
- It uses its **own alignment function**, separate from `skill.align_lemmas()` and from the evaluator's `align_lcs()`. Three implementations of one idea is two too many, and consolidating them is the clearest piece of remaining work in this part of the codebase.

14 User guide

Everything in this report is reproducible from the repository. This section is the operating manual: what to install, what order things run in, and what each command actually does.

14.1 Setup

Python 3.10 in a virtual environment. A CUDA GPU is needed for training and for generating model predictions; everything else runs on a CPU.

```
python -m venv venv
venv\Scripts\activate            # Windows PowerShell
pip install unsloth trl peft datasets bitsandbytes
pip install streamlit pyenchant reportlab
```

Requirement	Needed for	Notes
CUDA GPU, 4 GB+	training, and generating predictions	the reported run used an RTX 3050 Laptop (4 GB). One 3B model in 4-bit fits; two do not
MALINDO Morph CSV	every non-neural system, and the GUI	255,941 rows under `malindo_morph/`; ~40 s to load
pyenchant	English detection in the labeller	optional — the cascade degrades to `oov` without it
A YouTube Data API key	collecting new comments only	not needed to reproduce anything in this report; the corpus is already in `data/`

14.2 The pipeline, in order

Stages 1–3 have already been run and their outputs are committed. **To reproduce the report you only need stages 4 and 5**; the earlier ones are documented so the corpus can be rebuilt or extended.

#	Stage	Command	Produces
1	Collect comments	`python crawl/youtube_crawl.py --videos N`	`data/youtube_raw.jsonl` 21,291 comments
2	Filter to rojak sentences	`python crawl/build_corpus.py`	`data/corpus.jsonl` 10,099 sentences, banded
3	Silver-label and split	`python crawl/build_silver.py --n 8000`	`silver.jsonl`, `training_split.jsonl` (7,200), `dev_split.jsonl` (800)
3b	Build the gold test set	`python crawl/make_goldsheet.py --collect`	`data/gold.jsonl` 150 hand-corrected sentences
4	**Fine-tune**	`cd unsloth` `python llm.py --train`	`unsloth/lemmatizer_lora_yt/` 97.3 MB adapter
5	**Evaluate**	`python crawl/evaluate.py --model --base`	every table in section 12

Paths are relative to the **repository root** except where a `cd unsloth` is shown. `evaluate.py` resolves its own paths from `__file__`, so it runs from anywhere; `llm.py` does not, and must be run from inside `unsloth/`.

14.3 Training

```
cd unsloth
python llm.py --train
```

Defaults are set so a full run fits inside two hours on a 4 GB laptop GPU. A `plan:` line prints the whole schedule before any weights move, so an unreasonable configuration is visible in the first second rather than the ninetieth minute.

Flag	Default	What it does
`--epochs`	2	Both earlier runs picked their best checkpoint before the end of epoch 2, so the third only overfitted
`--budget-mins`	120	**Hard wall-clock cap.** Stops training when spent; the adapter saved is still the best checkpoint the dev set saw, not the last one. 0 disables
`--bs` / `--accum`	8 / 1	Effective batch 8. Sequences average 79 tokens, so a small batch leaves the GPU idle. On out-of-memory: `--bs 4 --accum 2`
`--eval-rows`	200	Dev rows scored *in the loop* to pick the checkpoint. Cannot change the trained weights — nothing backpropagates through an evaluation
`--eval-every`	100	Steps between evaluations, and between checkpoint writes (`load_best_model_at_end` requires them to match)

What a healthy run prints: a `plan:` line, then loss every 5 steps, an `eval_loss` every 100, and at the end the full dev curve with **BEST eval_loss … <- checkpoint kept**. If it says the best is the last step, the model was still improving and more epochs may help. If no `eval_loss` appears at all, read the `[config]` lines — they name every argument the installed TRL silently dropped.

14.4 Trying it on your own sentences

```
cd unsloth
python chat.py --adapter lemmatizer_lora_yt
```

An interactive prompt. Type a rojak sentence, get the lemma sentence back. The adapter is asked which prompt version it was trained with (`skill.prompt_for()`), so a v1 adapter can never be driven with a v2 prompt by accident.

Flag	What it does
`--adapter NAME`	which adapter folder under `unsloth/` to load
`--stages`	print every preprocessing stage, so you can see what the model was actually given
`--raw`	skip preprocessing entirely and send the sentence as typed
`--numbers POLICY`	what preprocessing does with digits (default: strip)
`--plain`	no colour — for redirecting output to a file

Two components run standalone with no GPU, which is what makes the non-neural half testable:

```
python preprocess.py --demo     # every cleaning stage, worked through
python skill.py --score         # the rule cascade against gold
```

14.5 Running the GUI

```
streamlit run app.py
```

Opens on `http://localhost:8501`. Run it from the **repository root**, not from `unsloth/`.

Step	What to do
1	Wait for “Loading MALINDO Morph…” — about 40 s the first time, then cached for the life of the server
2	Type a sentence and press **Lemmatise** (or Ctrl+Enter)
3	Read the token-by-token table: MALINDO, rules, and the rule tag that fired for each token
4	*Optional, needs a GPU:* tick **Enable (needs GPU)** in the sidebar to add the fine-tuned column. About 30 s to load, then cached

The model checkbox is **off by default**, so the page works on any machine. The rule columns always run; nothing the model does can take the page down. See section 13 for how it works and section 12.8 for why both systems are shown rather than one.

14.6 Reproducing the results tables

```
python crawl/evaluate.py --model --base    # one GPU pass, ~15 min
python crawl/evaluate.py --reuse           # re-score cache, no GPU
python crawl/evaluate.py --model --reuse-base   # only the adapter runs
```

Flag	When to use it
`--model`	generate predictions from the fine-tuned adapter (needs a GPU)
`--base`	also run the un-fine-tuned base model, for the before/after column
`--reuse`	score the cached predictions instead of regenerating. **Every table in section 12 comes back in about a minute with no GPU**
`--reuse-base`	score the base from cache while still generating the fine-tune — one model on the GPU at a time, which is what makes this fit in 4 GB
`--dev`	score the dev split. Development only: dev is silver-labelled, so `rules` scores ~100% there *by construction*

Two guards worth knowing about

Predictions are cached and the cache is checked against the adapter. The cache is keyed on the adapter *name*, so retraining in place would otherwise leave a complete-looking file full of the old adapter's answers. `--reuse` compares timestamps and refuses to score a cache older than the weights.

The split is verified, not assumed. `python crawl/verify_splits.py` re-derives 21 assertions from the files on disk — counts, disjointness of train/dev/gold by identifier, band stratification, and reproducibility from the seed. Run it after any change to the data.

14.7 Troubleshooting

Symptom	Cause	Fix
`Some modules are dispatched on the CPU or the disk`	A previous model is still resident. Two 3B models do not fit in 4 GB	Already handled by `_free_gpu()`; if it recurs, use `--reuse-base` so only one model loads
`can't open file …\unsloth\crawl\evaluate.py`	`crawl/` is relative to the repository root, and you are in `unsloth/`	`cd ..` first, or run `python ../crawl/evaluate.py`
`import unsloth` **finds nothing usable**	The repository has a folder called `unsloth/` which shadows the library	Handled by `_import_unsloth()`. Do not add an `__init__.py` to that folder
Triton cannot find a compiler	Windows Store Python reports a site-packages path that packages are not installed into	Handled by `_ensure_cc()`; or set `CC` to `triton/runtime/tcc/tcc.exe`
`Could not build the patched trl.trainer.grpo_trainer`	Unsloth failing to patch a trainer this project does not use	Harmless. Ignore it
Training is slower than expected	Corpus size drives steps, evaluation size and checkpoint count together	`--budget-mins` caps it outright; the `plan:` line shows the schedule before it starts

14.8 Where everything lives

Path	Contents
`crawl/`	collection, filtering, silver labelling, gold sheet, evaluation, split verification

Path	Contents
`unsloth/`	training (`llm.py`), inference (`chat.py`), preprocessing, the skill contract, and the adapters
`data/`	corpus, silver, the three splits, gold, the split manifest, and cached model predictions
`malindo_morph/`	MALINDO Morph 2024 and BM_dict
`app.py`	the Streamlit GUI (section 13)

15 Discussion and Conclusion

15.1 Discussion of findings

Three findings came out of the evaluation. They are separable, and a report that runs them together will misrepresent all three.

Finding 1 — fine-tuning transformed the model

The same Llama 3.2 3B, the same prompt, the same 150 test sentences. The only difference is a 97.3 MB adapter holding 24.3 M parameters — **0.76%** of the base model's weights.

Measure	Before	After	Change
Token accuracy	12.9%	93.8%	**+80.9 points**
F1 on the change decision	10.9%	90.0%	**+79.1 points**
Sentence exact match	0.7%	58.0%	**+57.3 points**
Format-failure rate	90.0%	6.7%	**–83.3 points**
Word error rate	2.1115	0.0660	**32× lower**

The base model scored **below the do-nothing floor** of 66.7%. It was not lemmatising badly — it was explaining, translating and conversing, which is what a WER above 2.0 means: more wrong words emitted than there were words to produce. Whatever else this project shows, it shows that a general-purpose 3B model cannot do this task from a prompt, and that a small adapter fixes that almost completely.

Finding 2 — the rule cascade is a genuinely strong baseline

95.1% accuracy, 92.0% F1, 64.7% of sentences perfect, no format failures, and it runs on a CPU at 1.3 µs per token. It is not a straw man, and it got stronger during the project: adding BM_dict to the labeller and correcting four wrong entries took it from 22.0% to 47.0% on the honest subset. Both systems were given the same lexicon, which is what makes the comparison fair — and it means the neural system had considerably more to beat at the end than at the start.

Finding 3 — the two are separated by 31 tokens, and that gap is not what it appears

Comparison	Result	Verdict
All 1,786 tokens	fine-tune wins 4, rules wins 27, p = 3.4e-05	**rules significantly ahead** — but by 31 tokens out of 1,786, a 1.3-point difference
132 honest tokens	fine-tune wins 2, rules wins 9, p = 0.065	**not significant** — statistically indistinguishable
Tokens both got wrong	83	**2.7× the size of the gap between them.** The shared blind spot is bigger than the difference

The finding behind the finding

The model was trained on **8,000 sentences whose every label was written by the rule cascade**. It is therefore **distilling** that cascade, and a student cannot systematically exceed a teacher who wrote every answer in the textbook.

Read that way, 93.8% against a 95.1% teacher is **98.6% of the teacher's performance**, reached by a general-purpose sequence model with no lexicon, no hand-written rules and 24.3 M trainable parameters. That is a successful distillation, and it is the correct frame for the whole comparison.

It also names the one change most likely to reverse the ranking: **train on human labels rather than rule labels**. The 132-token honest subset is the seed of exactly that dataset, and the fact that both systems fail over half of it shows the headroom is real.

15.2 Why the fine-tuned model is the chosen system

Section 12.8 records the selection against the criteria fixed before the numbers existed. This section gives the reasoning behind the choice. It rests on five arguments, and — stated plainly — **none of them is a claim to higher test-set accuracy**.

1. The test set is biased toward the rule system

The gold sheet was pre-filled by `propose()` and then corrected by hand. The annotator accepted the rule output on **1,654 of 1,786 tokens — 92.6% of the test set**. On that 92.6%, `rules` is being graded against its own suggestions. Its 95.1% is not a neutral measurement, and the gap to 93.8% is measured almost entirely on ground the rule system chose.

2. Where the bias is removed, the systems are indistinguishable

On the 132 tokens a human overrode — the only part of the test set independent of the rule cascade — the scores are 47.0% and 41.7%, and McNemar returns **p = 0.065**. Nine wins against two is roughly what a fair coin produces once in fifteen tries. The correct statement is that the two systems **cannot be told apart** on the only measure that is entitled to separate them.

3. The rule cascade cannot answer outside its lexicons

`propose()` is a closed lookup. A token matched by no particle rule, no slang entry, no BM_dict entry and no MALINDO row falls through to `oov` and is returned unchanged — the system has no mechanism for guessing. This is **measured, not hypothesised**: of the 70 tokens the cascade gets wrong on the honest subset, **41 (59%) are exactly this failure** — `skrng`, `prcya`, `tgkp`, `dprmudahkan` and 37 others left as typed because no lexicon held them (section 12.7). Across the wider corpus, 6,042 of 96,552 tokens (6.3%) resolve the same way.

The honest qualifier: **the fine-tuned model does not currently fix them either**, because it was trained on labels the cascade wrote and those labels said "leave it alone". The argument is therefore about *capability*, not present performance — a sequence model can learn that vowel-dropping is regular, a lookup table cannot, and realising that needs the human-labelled data of 15.6 item 1. A report should claim exactly that much and no more.

4. The two systems have different maintenance paths

Extending the cascade means a human hand-editing SLANG or BM_dict, one word at a time — and the four wrong BM_dict entries found during this project (`abg` → `adik` among them, teaching the wrong lemma 41 times) show how quietly that path fails. Extending the model means labelling more sentences and retraining, which is a process that scales with data rather than with expert hours.

5. It is what the project set out to build

The research question was whether a small open-weight LLM can be adapted to lemmatise Malay rojak. The answer is yes, by an enormous margin over its own starting point, and the artefact that answers it is the adapter. A rule cascade assembled from existing lexicons is the control condition, not the contribution.

> **What must not be written**
>
> **Do not write that the fine-tuned model is more accurate than the rule system.** It is not, on this test set: 93.8% against 95.1%, F1 90.0 against 92.0, and McNemar p = 3.4e-05 on the full set. The number is one table away and an examiner will find it.
>
> The defensible sentence is: *"the rule cascade scores higher on a test set it pre-filled; the two are statistically indistinguishable on the portion a human corrected; the fine-tuned model is deployed for coverage, maintainability and because it is the project's contribution, with the rule output retained as a live fallback."*

15.3 Strengths of the solution

- **The evaluation is honest by construction.** The single most important design decision in this project was noticing that gold had been pre-filled by the rule system, and reporting the 132-token subset separately. Most comparisons of this kind never find that problem, and quietly report the inflated number.
- **Six systems, not two.** The do-nothing floor (66.7%) and the plain-dictionary baseline (66.9%) are what make every other number interpretable, and they answer the two questions an examiner asks first.
- **Nine metrics covering different failure modes.** Accuracy hides over-correction; precision exposes it. Token measures hide malformed output; the format-failure rate and WER expose it. Aggregates hide sampling noise; McNemar and the confidence interval expose it.
- **The splits are verified, not assumed.** `verify_splits.py` re-derives 21 assertions from the files on disk — counts, disjointness by identifier, band stratification, reproducibility from the seed.
- **Reproducible end to end.** One command regenerates every table; predictions are cached, and the cache refuses to be scored if it is older than the weights that produced it.
- **The deployed system degrades safely.** The rule path always runs, so no GPU failure, no format failure and no missing adapter can leave the user without an answer.

15.4 Limitations

Limitation	Consequence	Severity
Training labels came from the rule system	The model is distilling the cascade and cannot systematically exceed it. This is the ceiling on the whole result, not a detail.	**High**
The honest subset is only 132 tokens	95% interval roughly ±8.5 points, so differences under ~12 points cannot be resolved. The observed gap is 5.3.	**High**
Gold was pre-filled by the rule system	Every measure outside the honest subset is biased toward `rules`. Disclosed everywhere it appears, but not removable after the fact.	**High**
One annotator, no agreement figure	Label consistency is unquantified. The `tau` → `tahu / tau` inconsistency found in 12.7 is direct evidence it matters.	Medium
A single training run, seed 3407	No variance estimate, so part of a 1.3-point difference could be run-to-run noise.	Medium
6.7% format-failure rate	One sentence in fifteen comes back mis-aligned. Survivable only because the rule output is a live fallback.	Medium
Corpus is YouTube comments from 9 channels	One register of one platform. Nothing here establishes performance on rojak SMS, WhatsApp or Twitter.	Medium
The GUI does not preprocess its input	`chat.py` does; a sentence can behave differently in the two front ends (13.7).	Low

15.5 Conclusion

This project set out to build a lemmatiser for Malay rojak — informal, code-switched Malay of the kind found in social media comments — by fine-tuning a small open-weight language model. It collected 21,291 YouTube comments, filtered them to 10,099 rojak sentences, labelled 8,000 of them through a nine-stage rule cascade, held out 800 for validation and 150 hand-corrected sentences for testing, and trained a QLoRA adapter over Llama 3.2 3B.

The fine-tuning succeeded. Token accuracy rose from 12.9% to 93.8%, F1 from 10.9% to 90.0%, and the rate at which the model failed to return a usable sentence fell from 90.0% to 6.7%. A 24.3 M parameter adapter — under 1% of the base model — was enough to convert a general-purpose chat model into a working lemmatiser.

It did not beat the rule baseline on this test set, scoring 93.8% against 95.1%. On the 132 tokens a human corrected, the two are statistically indistinguishable ($p = 0.065$). The most defensible explanation is also the

simplest: the model was trained on labels the rule cascade wrote, so it is distilling that cascade and reached 98.6% of it.

The system that is deployed and demonstrated

The lemmatiser is the fine-tuned model. Every lemma the system produces comes from the adapter; no rule output is merged into it, and no rule is consulted before or after generation. This is stated precisely because the interface also *displays* two other systems, and a reader could mistake a comparison view for a combined architecture.

	Component	What it contributes to the answer
The system	**Fine-tuned QLoRA adapter** `lemmatizer_lora_yt` over Llama 3.2 3B, v1 prompt contract	**Everything.** Greedy decoding, so the same sentence always gives the same answer in a demonstration
Display only	Rule cascade, `propose()`	**Nothing.** Shown in a parallel column with its per-token rule tags, so a viewer can see what a deterministic system would have answered. Its output is never substituted into the model's
Display only	MALINDO dictionary lookup	**Nothing.** A third column showing what an off-the-shelf Malay lemmatiser returns, which on rojak is close to nothing
Excluded	Base Llama 3.2 3B	Scores below the do-nothing floor and fails format 90% of the time. Reported as the before-fine-tuning baseline; not in the interface

Why this is not called a hybrid

A hybrid architecture combines two systems' predictions by some rule — voting, cascading, or one correcting the other. **Nothing of that kind happens here.** The three columns are computed independently and displayed together; the model's answer is identical whether or not the other two are shown.

There is exactly one place the rule output touches the model path, and it is presentational: when the model returns a different number of words than there were input tokens, `align_to_tokens()` uses the rule output as a reference sequence to decide which model word belongs on which table row. **It changes no lemma.** Unmatched rows are shown as a dash, not filled in from the rule column.

Calling it hybrid would invite the question "how are the two predictions combined?", and the honest answer is that they are not. *Fine-tuned model, with a rule-based comparison view* is both accurate and easier to defend.

The demonstrator is `app.py`, a Streamlit page run with `streamlit run app.py`, described in section 13. Showing the alternatives beside the answer is the honest presentation of a result where the systems are 5 points and one non-significant p-value apart — but the system being demonstrated is the model.

15.6 Future enhancements

Ordered by expected value, and each one is scoped to be achievable rather than aspirational.

Priority	Enhancement	Why it matters	Effort
1	**Train on human-corrected labels**	The single change that could reverse the ranking. While every training target is written by `propose()`, the model can only approximate it. The 132 honest tokens are the seed; 1,000–2,000 hand-labelled sentences would break the ceiling.	High — annotation time
2	**Expand the gold test set to 400–500 sentences**	At n = 132 the interval is ±8.5 points, so the deciding measure cannot resolve the 5.3-point gap it was built to test. Roughly tripling it would bring the resolution to ~5 points.	Medium

Priority	Enhancement	Why it matters	Effort
3	**Targeted training for affix boundaries**	The model's 27 losses are systematic, not random: over-eager prefix stripping (`manaikkan` → `manaik`, `pengantin` → `kantin`). Systematic errors respond to targeted examples.	Low
4	**Repeat training across 3–5 seeds**	Gives a variance estimate, without which a 1.3-point difference cannot be distinguished from run-to-run noise. Two hours per run under the current budget.	Low — compute only
5	**Second annotator on a 150-token overlap**	Produces an inter-annotator agreement figure, which every reviewer of a hand-labelled test set will ask for.	Low
6	**A context probe of constructed minimal pairs**	y → `yang` versus "42 y" is a real context failure of any per-token system, but only one form in 934 exhibits it here. A small constructed set would test the mechanism directly — reported as a diagnostic, never as a test-set result.	Low
7	**Deploy the v2 reasoning skill**	v2 emits a rule tag with every lemma, which would give the model the same token-by-token explainability the rule column already has — currently the rule path's main practical advantage in the GUI.	Medium
8	**Broaden the corpus beyond YouTube**	Nine channels of one platform is one register. Twitter or WhatsApp rojak would test whether any of this generalises.	High — new collection

Items 1 and 2 are the ones that would change the result. Everything else improves the evidence for a conclusion that is already supportable.